

## Chapter 22 – Struts

### *Model-View-Controller (MVC)*

---

Model-View-Controller (MVC) is a design pattern put together to help control change. MVC decouples interface from business logic and data.

- **Model:** The model contains the core of the application's functionality. The model encapsulates the state of the application. Sometimes the only functionality it contains is state. It knows nothing about the view or controller.
- **View:** The view provides the presentation of the model. It is the *look* of the application. The view can access the model getters, but it has no knowledge of the setters. In addition, it knows nothing about the controller. The view should be notified when changes to the model occur.
- **Controller:** The controller reacts to the user input. It creates and sets the model.

### *Struts Framework*

---

A framework is made up of the set of classes which allow us to use a library in a best possible way for a specific requirement. Struts framework is an open-source framework for developing the web applications in Java EE, based on MVC-2 architecture. It uses and extends the Java Servlet API. Struts is robust architecture and can be used for the development of application of any size. Struts framework makes it much easier to design scalable, reliable Web applications with Java.

### *Struts 2*

---

Struts2 is popular and mature web application framework based on the MVC design pattern. Struts2 is not just the next version of Struts 1, but it is a complete rewrite of the Struts architecture.

The Web Work framework started off with Struts framework as the basis and its goal was to offer an enhanced and improved framework built on Struts to make web development easier for the developers.

After some time, the Web work framework and the Struts community joined hands to create the famous Struts2 framework. The framework is designed to streamline the full development cycle, from building, to deploying, to maintaining applications over time. Apache Struts 2 was originally known as Web Work.

### Important Features of Struts 2:

1. POJO forms and POJO actions
2. Tag support - Improved and Less Coding
3. AJAX support
4. Easy Integration with other Frameworks
5. Template Support for generating views
6. Plugin Support
7. Integrated Profiling and Debugging Tools
8. Easy to modify tags using Free marker templates (Basic web knowledge is enough)
9. Promote less configuration using default values for various settings

### *Differences between Struts 1 and Struts 2*

---

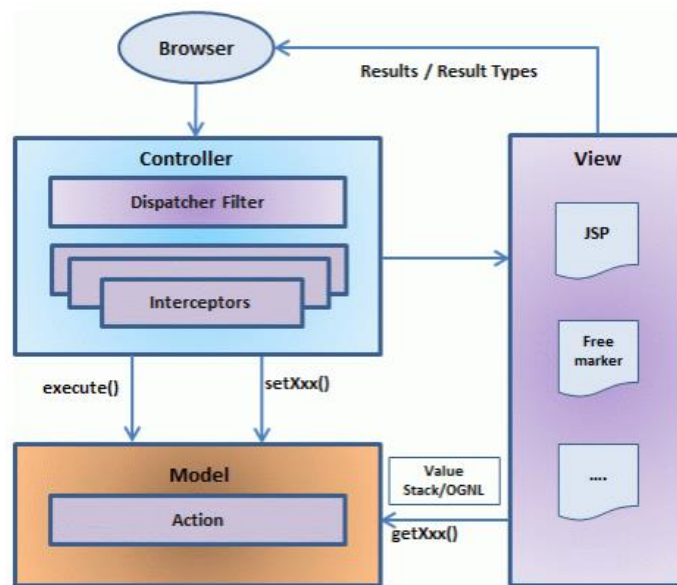
| Struts 1                                                                                  | Struts 2                                                                                  |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Action Class</b>                                                                       |                                                                                           |
| Abstract class                                                                            | Interface                                                                                 |
| <b>Threading Model</b>                                                                    |                                                                                           |
| Singleton and thread-safe                                                                 | New instance and no thread safety issues                                                  |
| <b>Servlet Dependency</b>                                                                 |                                                                                           |
| Depends on Servlet API                                                                    | No dependency on Servlet API                                                              |
| <b>Testability</b>                                                                        |                                                                                           |
| Unit test complexity                                                                      | DI makes unit testing easier                                                              |
| <b>Harvesting Input</b>                                                                   |                                                                                           |
| Redundant Action Forms                                                                    | Eliminates redundancy using Action properties. Supports Action Forms too.                 |
| <b>Expression Language</b>                                                                |                                                                                           |
| Integrates with JSTL & EL                                                                 | Integrates with JSTL & OGNL                                                               |
| <b>Binding Values into Views</b>                                                          |                                                                                           |
| Binds objects into the page context using JSP mechanism                                   | Decouples reusable views from values using Value Stack technology                         |
| <b>Type Conversion</b>                                                                    |                                                                                           |
| common-beanutils for type conversion                                                      | OGNL for type conversion                                                                  |
| <b>Validation</b>                                                                         |                                                                                           |
| Manual validation - validate method on the action form                                    | Xwork Validation framework in addition to manual validation                               |
| <b>Control of Action Execution</b>                                                        |                                                                                           |
| Each module- separate Request Processor(lifecycles) - shared by all Actions in the module | Different lifecycles created on a per Action basis via Interceptor Stacks (Custom Stacks) |

## Struts 2 Architecture

From a high level, Struts2 is a pull-MVC (or MVC2) framework. The Model-View-Controller pattern in Struts2 is realized with following five core components:

1. Actions
2. Interceptors
3. Value Stack / OGNL
4. Results / Result types
5. View technologies

Struts2 is slightly different from a traditional MVC framework in that the action takes the role of the model rather than the controller, although there is some overlap.



The above diagram depicts the Model, View and Controller to the Struts2 high level architecture. The controller is implemented with a Struts2 dispatch servlet filter as well as interceptors; the model is implemented with actions and the view as a combination of result types and results. The value stack and OGNL provide common thread, linking and enabling integration between the other components.

Apart from the above components, there will be a lot of information that relates to configuration. Configuration for the web application, as well as configuration for actions, interceptors, results, etc.

### Request Life Cycle

Based on the above diagram, one can explain the user's request life cycle in Struts 2 as follows:

1. User sends a request to the server for requesting for some resource (i.e. pages).
2. The Filter Dispatcher looks at the request and then determines the appropriate Action.
3. Configured interceptors functionalities apply such as validation, file upload etc.
4. Selected action is executed to perform the requested operation.
5. Again, configured interceptors are applied to do any post-processing if required.
6. Finally the result is prepared by the view and returns the result to the user.

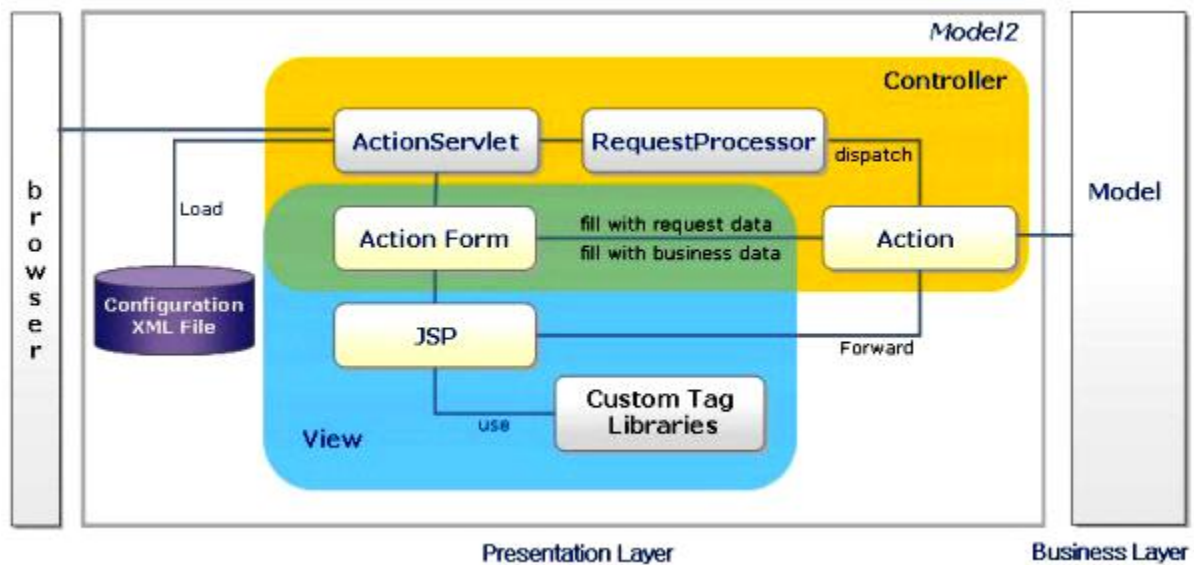
## Struts 1 Architecture

Struts components can be categorized into Model, View and Controller:

- **Model:** Components like business logic /business processes and data are the part of model.
- **View:** HTML, JSP are the view components.
- **Controller:** Action Servlet of Struts is part of Controller components which works as front controller to handle all the requests.

Struts is a set of cooperating classes, servlets, and JSP tags that make up a reusable MVC 2 design.

- JavaBeans components for managing application state and behavior.
- Event-driven development (via listeners as in traditional GUI development).
- Pages that represent MVC-style views; pages reference view roots via the JSF component tree.



### Request Life Cycle

Executed in the following order when request is sent by client

1. When web application starts, Struts configuration file (struts-config.xml) loading is performed using Action Servlet set in web.xml.
2. Decide Action Mapping according to request from Request Processor defined in struts-config.xml.
3. Execute execute() method of Action class according to URL and action mapping information defined in struts-config.xml.
4. Action execute() method calls business logic by connecting to business layer.
5. Return Action Forward according to business logic execution result, and Controller performs forwarding with the appropriate view according to return value.

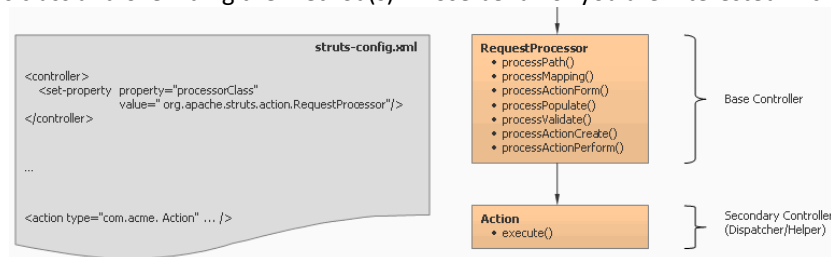
## ActionServlet

ActionServlet is a simple servlet which is the backbone of all Struts applications. It is the main Controller component that handles client requests and determines which Action will process each received request. It serves as an Action factory that helps to create specific Action classes based on user's request.

```
<servlet>                                web.xml
  <servlet-name>action</servlet-name>
  <servlet-class>
    org.apache.struts.action.ActionServlet
  </servlet-class>
  <init-param>
    <param-name>config</param-name>
    <param-value>
      /WEB-INF/struts-config.xml
    </param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

## RequestProcessor

RequestProcessor contains the processing logic that the Struts controller servlet performs as it receives each servlet request from the container. You can customize the request processing behavior by subclassing this class and overriding the method(s) whose behavior you are interested in changing.



## Action

An Action Class performs a role of an adapter between the contents of an incoming HTTP request and the corresponding business logic that should be executed to process this request.

```
public ActionForward execute(ActionMapping mapping, ActionForm form, HttpServletRequest req, HttpServletResponse res)
    throws Exception;
```

The different kinds of actions in Struts are ForwardAction, IncludeAction, DispatchAction, LookupDispatchAction and SwitchAction.

## ActionMapping

ActionMapping contains all the deployment information for a particular Action bean. This class is to determine where the results of the Action will be sent once its processing is complete.

## ActionForm

ActionForm is a JavaBean which represents the form inputs containing the request parameters from the View referencing the Action bean. The important methods of ActionForm are `validate()` and `reset()`.

```
public ActionErrors validate(ActionMapping mapping, HttpServletRequest request)
```

The lifecycle of ActionForm invoked by the RequestProcessor is as follows:

- Retrieve or Create Form Bean associated with Action
- "Store" FormBean in appropriate scope (request or session)
- Reset the properties of the FormBean
- Populate the properties of the FormBean

- Validate the properties of the FormBean
- Pass FormBean to Action

### DynaActionForm

It's a specialized subclass of ActionForm that allows the creation of form beans with dynamic sets of properties (configured in configuration file), without requiring the developer to create a Java class for each type of form bean.

```
<form-bean name="loginForm" type="org.apache.struts.action.DynaActionForm" >
  <form-property name="userName" type="java.lang.String"/>
  <form-property name="password" type="java.lang.String" />
</form-bean>
```

### ActionErrors

A class that encapsulates the error messages being reported by the validate() method of an ActionForm. Validation errors are either global to the entire ActionForm bean they are associated with, or they are specific to a particular bean property. Each individual error is described by an ActionMessage object, which contains a message key.

### ActionForward

An ActionForward represents a destination to which the controller, RequestProcessor, might be directed to perform a RequestDispatcher.forward or HttpServletResponse.sendRedirect to, as a result of processing activities of an Action class.

### Struts Tag Libraries

The various Struts tag libraries are HTML Tags, Bean Tags, Logic Tags, Template Tags, Nested Tags and Tiles Tags.

### Struts Configuration File Structure

```
<struts-config>
  <form-beans>
    <form-bean name="CustomerForm"
      type="mybank.example.CustomerForm"/>

    <form-bean name="LogonForm"
      type="mybank.example.LogonForm"/>
  </form-beans>
  <global-forwards>
    <forward name="logon" path="/logon.jsp"/>
    <forward name="logoff" path="/logoff.do"/>
  </global-forwards>
  <action-mappings>
    <action path="/submitDetailForm"
      type="mybank.example.CustomerAction"
      name="CustomerForm"
      scope="request"
      validate="true"
      input="/CustomerDetailForm.jsp">
      <forward name="success"
        path="/ThankYou.jsp"
        redirect="true" />
      <forward name="failure"
        path="/Failure.jsp" />
    </action>
    <action path="/logoff" parameter="/logoff.jsp"
      type="org.apache.struts.action.ForwardAction" />
  </action-mappings>
  <controller
    processorClass="org.apache.struts.action.RequestProcessor" />
  <message-resources parameter="mybank.ApplicationResources"/>
</struts-config>
```

*Form bean Definitions*

*Global Forward Definitions*

*Action Mappings*

*Controller Configuration*

*Message Resource Definition*

# Chapter 23 – Spring

## *Inversion of Control (IOC)*

---

The basic concept of the Inversion of Control pattern (also known as dependency injection) is that you do not create your objects but describe how they should be created. You don't directly connect your components and services together in code but describe which services are needed by which components in a configuration file. A container (in the case of the Spring framework, the IOC container) is then responsible for hooking it all up.

i.e., Applying IoC, objects are given their dependencies at creation time by some external entity that coordinates each object in the system. That is, dependencies are injected into objects. So, IoC means an inversion of responsibility with regard to how an object obtains references to collaborating objects.

**Hollywood Principle - Don't call me, I will call you**

### **Different Types of IOC**

There are three types of dependency injection:

- **Constructor Injection** (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- **Setter Injection** (e.g. Spring): Dependencies are assigned through JavaBean properties (ex: setter methods).
- **Interface Injection** (e.g. Avalon): Injection is done through an interface.

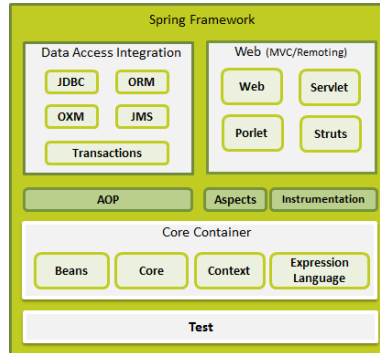
*Note: Spring supports only Constructor and Setter Injection*

### **Benefits of IOC (Dependency Injection)**

Benefits of IOC are as follows:

- Minimizes the amount of code in your application.
- Make your application more testable by not requiring any singletons or JNDI lookup mechanisms in your unit test cases.
- Loose coupling is promoted with minimal effort and least intrusive mechanism.
- IOC containers support eager instantiation and lazy loading of services.

Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development.



## Advantages of Spring

The advantages of Spring framework are as follows:

- Spring has layered architecture. Use what you need and leave you don't need now.
- Spring enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
- Dependency Injection and Inversion of Control simplifies JDBC
- Open source and no vendor lock-in.

## Features of Spring

- **Lightweight:**  
Spring is lightweight when it comes to size and transparency. The basic version of spring framework is around 1MB. And the processing overhead is also very negligible.
- **Inversion of control (IOC):**  
Loose coupling is achieved in spring using the technique Inversion of Control. The objects give their dependencies instead of creating or looking for dependent objects.
- **Aspect oriented (AOP):**  
Spring supports Aspect oriented programming and enables cohesive development by separating application business logic from system services.
- **Container:**  
Spring contains and manages the life cycle and configuration of application objects.
- **MVC Framework:**  
Spring comes with MVC web application framework, built on core Spring functionality. This framework is highly configurable via strategy interfaces, and accommodates multiple view technologies like JSP, Velocity, Tiles, iText, and POI. But other frameworks can be easily used instead of Spring MVC Framework.
- **Transaction Management:**  
Spring framework provides a **generic abstraction layer** for transaction management. This allows the developer to add the pluggable transaction managers, and making it easy to demarcate

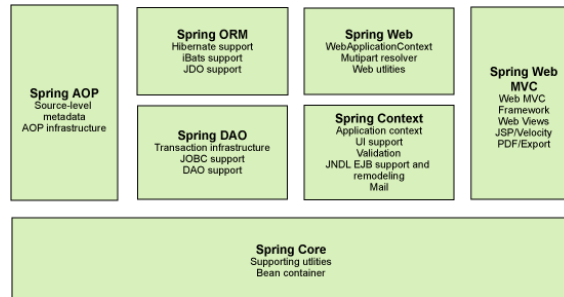
transactions without dealing with low-level issues. Spring's transaction support is not tied to J2EE environments and it can be also used in container less environments.

- **JDBC Exception Handling:**

The JDBC abstraction layer of the Spring offers a meaningful **exception hierarchy**, which simplifies the error handling strategy. Integration with Hibernate, JDO, and iBATIS: Spring provides best Integration services with Hibernate, JDO and iBATIS

**Seven Modules:**

1. Spring Core - IOC-DI/Bean Factory Container
2. Spring AOP – Cross cutting concerns such as logging, auditing, transactions, security and caching
3. Spring MVC – Model View Controller
4. Spring ORM – Integration with Hibernate, iBatis and JDO
5. Spring DAO - JDBC abstraction layer and exception handling
6. Spring Web - Web oriented integration (Web Application Context and Multipart Resolver)
7. Spring Context – Application Context, UI support, Validation, JNDI, EJB and Mail support



**Bean Definition**

The objects that form the backbone of your application and that are managed by the Spring IoC container are called **beans**. A bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container. These beans are created with the configuration metadata that you supply to the container, for example, in the form of XML <bean/> definitions which you have already seen in previous chapters.

The bean definition contains the information called **configuration metadata** which is needed for the container to know the followings:

- How to create a bean
- Bean's lifecycle details
- Bean's dependencies

All the above configuration metadata translates into a set of the following properties that make up each bean definition.

| Properties               | Description                                                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| class                    | This attribute is mandatory and specify the bean class to be used to create the bean.                                                                                |
| name                     | This attribute specifies the bean identifier uniquely. In XML-based configuration metadata, you use the id and/or name attributes to specify the bean identifier(s). |
| scope                    | This attribute specifies the scope of the objects created from a particular bean definition and it will be discussed in bean scopes chapter.                         |
| constructor-arg          | This is used to inject the dependencies using constructor arguments.                                                                                                 |
| properties               | This is used to inject the dependencies using the setter methods.                                                                                                    |
| autowiring mode          | This is used to inject the dependencies in auto-wiring mode (autowire)                                                                                               |
| lazy-initialization mode | A lazy-initialized bean tells the IoC container to create a bean instance when it is first requested, rather than at startup (lazy-init)                             |
| initialization method    | A callback to be called just after all necessary properties on the bean has been set by the container. It will be discussed in bean life cycle chapter (init-method) |
| destruction method       | A callback to be used when the container containing the bean is destroyed. It will be discussed in bean life cycle chapter (destroy-method)                          |

## Bean Scopes

The Spring Framework supports following five scopes, three of which are available only if you use a web-aware Application Context (Web Application only). The first two scopes can be used for both stand-alone and web applications.

| Scope     | Description                                                                                                                                                                                                                                                                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| singleton | This scopes the bean definition to a single instance per Spring IoC container (default). This single instance is stored in a cache of such singleton beans, and all subsequent requests and references for that named bean return the cached object.                                                                                             |
| prototype | This scopes a single bean definition to have any number of object instances. If scope is set to prototype, the Spring IoC container creates new bean instance of the object every time a request for that specific bean is made. <b>As a rule, use the prototype scope for all state-full beans and the singleton scope for stateless beans.</b> |
| request   | This scopes a bean definition to an HTTP request. Only valid in the context of a web-aware Spring Application Context.                                                                                                                                                                                                                           |
| session   | This scopes a bean definition to an HTTP session. Only valid in the context of a web-aware Spring Application Context.                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| global-session | This scopes a bean definition to a global HTTP session. Only valid in the context of a web-aware Spring Application Context. Typically only valid when used in a <b>portlet context</b> . |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Spring Configuration Metadata

Spring IoC container is totally decoupled from the format in which this configuration metadata is actually written. There are following three important methods to provide configuration metadata to the Spring Container:

1. XML-based configuration file (Declarative)
2. Annotation-based configuration (Declarative)
3. Java-based configuration (Programmatic)

## Bean Life Cycle

The life cycle of a Spring bean is easy to understand. When a bean is instantiated, it may be required to perform some initialization to get it into a usable state. Similarly, when the bean is no longer required and is removed from the container, some cleanup may be required.

Though, there is a list of the activities that take place behind the scenes between the time of bean Instantiation and its destruction, but this chapter will discuss only two important bean lifecycle callback methods which are required at the time of bean initialization and its destruction.

To define setup and teardown for a bean, we simply declare the `<bean>` with **init-method** and/or **destroy-method** parameters. The `init-method` attribute specifies a method that is to be called on the bean immediately upon instantiation. Similarly, `destroy-method` specifies a method that is called just before a bean is removed from the container.

## Initialization callbacks:

The *org.springframework.beans.factory*.**InitializingBean** interface specifies a single method.

```
void afterPropertiesSet() throws Exception;
```

So you can simply implement above interface and initialization work can be done inside `afterPropertiesSet()` method as follows:

```
public class ExampleBean implements InitializingBean {
    public void afterPropertiesSet() {
        // do some initialization work
    }
}
```

In the case of XML-based configuration metadata, you can use the **init-method** attribute to specify the name of the method that has a void no-argument signature. For example:

```
<bean id="exampleBean" class="examples.ExampleBean" init-method="initialize"/>
```

Following is the class definition:

```
public class ExampleBean {
    public void initialize() {
        // do some initialization work
    }
}
```

## Destruction callbacks

The *org.springframework.beans.factory*.**DisposableBean** interface specifies a single method.

```
void destroy() throws Exception;
```

So you can simply implement above interface and finalization work can be done inside `destroy()` method as follows:

```
public class ExampleBean implements DisposableBean {
    public void destroy() {
        // do some destruction work
    }
}
```

In the case of XML-based configuration metadata, you can use the **destroy-method** attribute to specify the name of the method that has a void no-argument signature. For example:

```
<bean id="exampleBean" class="examples.ExampleBean" destroy-method="dispose"/>
```

Following is the class definition:

```
public class ExampleBean {
    public void dispose() {
        // do some destruction work
    }
}
```

If you are using Spring's IoC container in a non-web application environment; for example, in a rich client desktop environment; you register a shutdown hook with the JVM. Doing so ensures a graceful shutdown and calls the relevant destroy methods on your singleton beans so that all resources are released.

**It is recommended that you do not use the InitializingBean or DisposableBean callbacks, because XML configuration gives much flexibility in terms of naming your method.**

## Bean Post Processor

The **BeanPostProcessor** interface defines callback methods that you can implement to provide your own **instantiation logic, dependency-resolution logic** etc. You can also implement some custom logic after the Spring container finishes instantiating, configuring, and initializing a bean by plugging in one or more BeanPostProcessor implementations.

You can configure multiple BeanPostProcessor interfaces and you can control the order in which these BeanPostProcessor interfaces execute by setting the **order** property provided the BeanPostProcessor implements the **Ordered** interface.

The BeanPostProcessors operate on bean (or object) instances which means that the Spring IoC container instantiates a bean instance and then BeanPostProcessor interfaces do their work.

An **ApplicationContext** automatically detects any beans that are defined with implementation of the **BeanPostProcessor** interface and registers these beans as post-processors, to be then called appropriately by the container upon bean creation.

```
public class InitHelloWorld implements BeanPostProcessor {

    public Object postProcessBeforeInitialization(Object bean,
        String beanName) throws BeansException {
        System.out.println("BeforeInitialization : " + beanName);
        return bean; // you can return any other object as well
    }

    public Object postProcessAfterInitialization(Object bean,
        String beanName) throws BeansException {
        System.out.println("AfterInitialization : " + beanName);
        return bean; // you can return any other object as well
    }

}
```

```
AbstractApplicationContext context =
    new ClassPathXmlApplicationContext("Beans.xml");

HelloWorld obj = (HelloWorld) context.getBean("helloWorld");
obj.getMessage();
context.registerShutdownHook();
```

```
<bean id="helloWorld" class="com.mesonsoft.HelloWorld"
    init-method="init" destroy-method="destroy">
    <property name="message" value="Hello World!"/>
</bean>

<bean class="com.mesonsoft.InitHelloWorld" />
```

## Bean Definition Inheritance

A bean definition can contain a lot of configuration information, including constructor arguments, property values, and container-specific information such as initialization method, static factory method name, and so on.

A child bean definition inherits configuration data from a parent definition. The child definition can override some values, or add others, as needed.

Spring Bean definition inheritance has nothing to do with Java class inheritance but inheritance concept is same. You can define a parent bean definition as a template and other child beans can inherit required configuration from the parent bean.

When you use XML-based configuration metadata, you indicate a child bean definition by using the **parent** attribute, specifying the parent bean as the value of this attribute.

```
<bean id="helloWorld" class="com.mesonsoft.HelloWorld">
  <property name="message1" value="Hello World!"/>
  <property name="message2" value="Hello Second World!"/>
</bean>

<bean id="helloIndia" class="com.mesonsoft.HelloIndia"
  parent="helloWorld">
  <property name="message1" value="Hello India!"/>
  <property name="message3" value="Namaste India!"/>
</bean>
```

### Bean Definition Template

You can create a Bean definition template which can be used by other child bean definitions without putting much effort. While defining a Bean Definition Template, you should not specify **class** attribute and should specify **abstract** attribute with a value of **true** as shown below:

```
<bean id="beanTemplate" abstract="true">
  <property name="message1" value="Hello World!"/>
  <property name="message2" value="Hello Second World!"/>
  <property name="message3" value="Namaste India!"/>
</bean>

<bean id="helloIndia" class="com.mesonsoft.HelloIndia"
  parent="beanTemplate">
  <property name="message1" value="Hello India!"/>
  <property name="message3" value="Namaste India!"/>
</bean>
```

### Injection Inner Beans

As you know Java inner classes are defined within the scope of other classes, similarly, **inner beans** are beans that are defined within the scope of another bean. Thus, a `<bean/>` element inside the `<property/>` or `<constructor-arg/>` elements is called inner bean and it is shown below.

```
<bean id="outerBean" class="...">
  <constructor-arg>
    <bean id="innerBean" class="..." /> //CONSTRUCTOR INJECTION
  </constructor-arg>
  <property name="target">
    <bean id="innerBean" class="..." /> //SETTER INJECTION
  </property>
</bean>
```

### Injecting Collection

You have seen how to configure **primitive data type** using **value** attribute and **object references** using **ref** attribute of the `<property>` tag in your Bean configuration file. Both the cases deal with passing singular value to a bean.

Now what about if you want to pass plural values like Java Collection types **List**, **Set**, **Map**, and **Properties**. To handle the situation, Spring offers four types of collection configuration elements which are as follows:

| Element | Description                                                                                                    |
|---------|----------------------------------------------------------------------------------------------------------------|
| <list>  | This helps in wiring i.e. injecting a list of values, <b>allowing duplicates</b> .                             |
| <set>   | This helps in wiring a set of values but <b>without any duplicates</b> .                                       |
| <map>   | This can be used to inject a collection of name-value pairs where name and value can be of <b>any type</b> .   |
| <props> | This can be used to inject a collection of name-value pairs where the name and value are <b>both Strings</b> . |

You can use either <list> or <set> to wire any implementation of java.util.**Collection** or an **array**.

```

<!-- Definition for javaCollection -->
<bean id="javaCollection" class="com.mesonsoft.JavaCollection">
  <!-- results in a setAddressList(java.util.List) call -->
  <property name="addressList">
    <list>
      <value>INDIA</value>
      <value>USA</value>
      <value>USA</value>
    </list>
  </property>

  <!-- results in a setAddressSet(java.util.Set) call -->
  <property name="addressSet">
    <set>
      <value>INDIA</value>
      <value>USA</value>
      <value>USA</value>
    </set>
  </property>

  <!-- results in a setAddressMap(java.util.Map) call -->
  <property name="addressMap">
    <map>
      <entry key="1" value="INDIA"/>
      <entry key="3" value="USA"/>
      <entry key="4" value="USA"/>
    </map>
  </property>

  <!-- results in a setAddressProp(java.util.Properties) call -->
  <property name="addressProp">
    <props>
      <prop key="one">INDIA</prop>
      <prop key="three">USA</prop>
      <prop key="four">USA</prop>
    </props>
  </property>
</bean>

```

## Spring Beans Auto-Wiring

You have learnt how to declare beans using the `<bean>` element and inject `<bean>` with using `<constructor-arg>` and `<property>` elements in XML configuration file.

The Spring container can **autowire** relationships between collaborating beans without using `<constructor-arg>` and `<property>` elements which helps cut down on the amount of XML configuration you write for a big Spring based application.

### Autowiring Modes:

There are following autowiring modes which can be used to instruct Spring container to use autowiring for dependency injection. You use the **autowire** attribute of the `<bean/>` element to specify autowire mode for a bean definition.

| Mode        | Description                                                                                                                                                                                                                                                                                                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| no          | This is default setting which means no autowiring and you should use explicit bean reference for wiring. You have nothing to do special for this wiring. This is what you already have seen in Dependency Injection chapter.                                                                                                                                                      |
| byName      | Autowiring by property name. Spring container looks at the properties of the beans on which <i>autowire</i> attribute is set to <i>byName</i> in the XML configuration file. It then tries to match and wire its properties with the beans defined by the same names in the configuration file.                                                                                   |
| byType      | Autowiring by property datatype. Spring container looks at the properties of the beans on which <i>autowire</i> attribute is set to <i>byType</i> in the XML configuration file. It then tries to match and wire a property if its <b>type</b> matches with exactly one of the beans name in configuration file. If more than one such beans exists, a fatal exception is thrown. |
| constructor | Similar to <i>byType</i> , but type applies to constructor arguments. If there is not exactly one bean of the constructor argument type in the container, a fatal error is raised.                                                                                                                                                                                                |
| autodetect  | Spring first tries to wire using autowire by <b>constructor</b> , if it does not work, Spring tries to autowire by <b>byType</b> .                                                                                                                                                                                                                                                |

You can use **byType** or **constructor** autowiring mode to wire **arrays** and other typed-collections.

### Limitations with autowiring:

Autowiring works best when it is used consistently across a project. If autowiring is not used in general, it might be confusing to developers to use it to wire only one or two bean definitions. Though, autowiring can significantly reduce the need to specify properties or constructor arguments but you should consider the limitations and disadvantages of autowiring before using them.

| Limitations            | Description                                                                                                                                                      |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overriding possibility | You can still specify dependencies using <code>&lt;constructor-arg&gt;</code> and <code>&lt;property&gt;</code> settings which will always override auto-wiring. |

|                      |                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------|
| Primitive data types | You cannot autowire so-called simple properties such as primitives, Strings, and Classes.    |
| Confusing nature     | Auto-wiring is less exact than explicit wiring, so if possible prefer using explicit wiring. |

### Annotation Based Configuration

Starting from Spring 2.5, it became possible to configure the dependency injection using **annotations**. So instead of using XML to describe a bean wiring, you can move the bean configuration into the component class itself by using annotations on the relevant class, method, or field declaration.

Annotation injection is performed before XML injection, thus the latter configuration will override the former for properties wired through both approaches.

Annotation wiring is not turned on in the Spring container by default. So, before we can use annotation-based wiring, we will need to enable it in our Spring configuration file. So, consider to have following configuration file, in case you want to use any annotation in your Spring application.

```
<context:annotation-config/>
<!-- bean definitions go here -->
</beans>
```

Once **<context:annotation-config/>** is configured, you can start annotating your code to indicate that Spring should automatically wire values into properties, methods, and constructors. Let us see few important annotations to understand how they work:

| S.N. | Annotation & Description                                                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <u>@Required</u><br>The @Required annotation applies to bean property setter methods.                                                                              |
| 2    | <u>@Autowired</u><br>The @Autowired annotation can apply to bean property setter methods, non-setter methods, constructor and properties.                          |
| 3    | <u>@Qualifier</u><br>The @Qualifier annotation along with @Autowired can be used to remove the confusion by specifying which exact bean will be wired.             |
| 4    | <u>JSR-250 Annotations</u><br>Spring supports JSR-250 based annotations which include <b>@Resource</b> , <b>@PostConstruct</b> and <b>@PreDestroy</b> annotations. |

## Java Based Configuration

Java based configuration option enables you to write most of your Spring configuration without XML but with the help of few Java-based annotations explained below.

### @Configuration & @Bean Annotations:

Annotating a class with the **@Configuration** indicates that the class can be used by the Spring IoC container as a source of bean definitions. The **@Bean** annotation tells Spring that a method annotated with **@Bean** will return an object that should be registered as a bean in the Spring application context.

### Lifecycle Callbacks:

The **@Bean** annotation supports specifying arbitrary initialization and destruction callback methods, much like Spring XML's `init-method` and `destroy-method` attributes on the bean element

### Specifying Bean Scope:

The default scope is singleton, but you can override this with the **@Scope** annotation as given below.

### The @Import Annotation:

The **@Import** annotation allows for loading **@Bean** definitions from another configuration class. Consider a **ConfigA** class as given below.

```
@Configuration
@Import(ConfigA.class)
@ImportResource("spring.xml")
public class HelloWorldConfig {

    @Bean
    @Scope("prototype")
    public HelloWorld helloWorld(){
        return new HelloWorld();
    }

    @Bean(initMethod = "initialize", destroyMethod = "cleanup")
    public Foo foo() {
        return new Foo();
    }

    public class Foo {
        public void initialize() {
            // initialization logic
        }
        public void cleanup() {
            // destruction logic
        }
    }
}
```

Above Java code will be equivalent to the following XML configuration:

```
<bean id="helloWorld" class="com.mesonsoft.HelloWorld" scope="prototype" init-method="initialize" destroy-method="cleanup" />
```

## Injecting Bean Dependencies:

When @Beans have dependencies on one another, expressing that dependency is as simple as having one bean method calling another as follows:

```
@Configuration
public class AppConfig {
    @Bean
    public Foo foo() {
        Foo foo = new Foo (bar()); // CONSTRUCTOR INJECTION
        Foo foo = new Foo ();
        foo.setBar(bar());        // SETTER INJECTION
        return foo;
    }
    @Bean
    public Bar bar() {
        return new Bar();
    }
}
```

## Event Handling in Spring

The core of Spring framework is the **ApplicationContext**, which manages complete life cycle of the beans. The **ApplicationContext** publishes certain types of events when loading the beans. For example, a *ContextStartedEvent* is published when the context is started and *ContextStoppedEvent* is published when the context is stopped.

Event handling in the *ApplicationContext* is provided through the **ApplicationEvent** class and **ApplicationListener** interface. So if a bean implements the **ApplicationListener**, then every time an **ApplicationEvent** gets published to the **ApplicationContext**, that bean is notified.

Spring provides the following standard events:

| S.N. | Spring Built-in Events & Description                                                                                                                                                                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>ContextRefreshedEvent</b><br>This event is published when the <i>ApplicationContext</i> is either initialized or refreshed. This can also be raised using the <b>refresh()</b> method on the <b>ConfigurableApplicationContext</b> interface.                                              |
| 2    | <b>ContextStartedEvent</b><br>This event is published when the <i>ApplicationContext</i> is started using the <b>start()</b> method on the <b>ConfigurableApplicationContext</b> interface. You can poll your database or you can restart any stopped application after receiving this event. |
| 3    | <b>ContextStoppedEvent</b><br>This event is published when the <i>ApplicationContext</i> is stopped using the <b>stop()</b> method on the <b>ConfigurableApplicationContext</b> interface.                                                                                                    |
| 4    | <b>ContextClosedEvent</b><br>This event is published when the <i>ApplicationContext</i> is closed using the <b>close()</b> method on the <b>ConfigurableApplicationContext</b> interface. A closed context reaches its end of life; it cannot be refreshed or restarted.                      |
| 5    | <b>RequestHandledEvent</b><br>This is a web-specific event telling all beans that an HTTP request has been serviced.                                                                                                                                                                          |

Spring's event handling is **single-threaded** so if an event is published, until and unless all the receivers get the message, the processes are blocked and the flow will not continue. Hence, care should be taken when designing your application if event handling is to be used.

### Listening to Context Events:

To listen a context event, a bean should implement the **ApplicationListener** interface which has just one method **onApplicationEvent()**. So let us write an example to see how the events propagates and how you can put your code to do required task based on certain events.

```
public class CStartEventHandler
    implements ApplicationListener<ContextStartedEvent>{

    public void onApplicationEvent(ContextStartedEvent event) {
        System.out.println("ContextStartedEvent Received");
    }
}
```

```
ConfigurableApplicationContext context =
    new ClassPathXmlApplicationContext("Beans.xml");

// Let us raise a start event.
context.start();

HelloWorld obj = (HelloWorld) context.getBean("helloWorld");

obj.getMessage();

// Let us raise a stop event.
context.stop();
```

```
<bean id="cStartEventHandler" class="com.mesonsoft.CStartEventHandler"/>
```

### Custom Events in Spring

There are number of steps to be taken to write and publish your own custom events. Follow the instructions given below to **write**, **publish** and **handle** Custom Spring Events.

| Step | Description                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Create an event class, <i>CustomEvent</i> by extending <b>ApplicationEvent</b> . This class must define a default constructor which should <b>inherit</b> constructor from <i>ApplicationEvent</i> class.                                                                                                                                                                                         |
| 2    | Once your event class is defined, you can publish it from any class, let us say <i>CustomEventPublisher</i> which implements <b>ApplicationEventPublisherAware</b> . You will also need to declare these classes in XML configuration file as a bean so that the container can identify the bean as an event publisher because it implements the <i>ApplicationEventPublisherAware</i> interface. |
| 3    | A published event can be handled in a class, let us say <i>CustomEventHandler</i> which implements <b>ApplicationListener</b> interface and implements <b>onApplicationEvent</b> method for the custom event.                                                                                                                                                                                     |

```
public class CustomEvent extends ApplicationEvent{

    public CustomEvent(Object source) {
        super(source);
    }

    public String toString(){
        return "My Custom Event";
    }
}
```

```
public class CustomEventPublisher
    implements ApplicationEventPublisherAware {

    private ApplicationEventPublisher publisher;

    public void setApplicationEventPublisher
        (ApplicationEventPublisher publisher){
        this.publisher = publisher;
    }

    public void publish() {
        CustomEvent ce = new CustomEvent(this);
        publisher.publishEvent(ce);
    }
}
```

```
public class CustomEventHandler
    implements ApplicationListener<CustomEvent>{

    public void onApplicationEvent(CustomEvent event) {
        System.out.println(event.toString());
    }
}
```

```
CustomEventPublisher cvp = (CustomEventPublisher) context.getBean("customEventPublisher");
cvp.publish();
```

```
<bean id="customEventHandler" class="com.mesonsoft.CustomEventHandler"/>
<bean id="customEventPublisher" class="com.mesonsoft.CustomEventPublisher"/>
```

## AOP with Spring Framework

One of the key components of Spring Framework is the **Aspect oriented programming (AOP)** framework. Aspect Oriented Programming entails breaking down program logic into distinct parts called so-called concerns. The functions that span multiple points of an application are called **cross-cutting concerns** and these cross-cutting concerns are conceptually separate from the application's business logic. There are various common good examples of aspects like **logging, auditing, declarative transactions, security, and caching** etc.

**The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect.** Dependency Injection helps you decouple your application objects from each other and AOP helps you decouple cross-cutting concerns from the objects that they affect.

Spring AOP module provides **interceptors** to intercept an application, for example, when a method is executed, you can add extra functionality before or after the method execution.

### AOP Terminologies:

Before we start working with AOP, let us become familiar with the AOP concepts and terminology. These terms are not specific to Spring, rather they are related to AOP.

| Terms         | Description                                                                                                                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aspect        | A module which has a set of APIs providing cross-cutting requirements. For example, a logging module would be called AOP aspect for logging. An application can have any number of aspects depending on the requirement. |
| Join point    | This represents a point in your application where you can plug-in AOP aspect. You can also say, it is the actual place in the application where an action will be taken using Spring AOP framework.                      |
| Advice        | This is the actual action to be taken either before or after the method execution. This is actual piece of code that is invoked during program execution by Spring AOP framework.                                        |
| Pointcut      | This is a set of one or more join points where an advice should be executed. You can specify pointcuts using expressions or patterns.                                                                                    |
| Introduction  | An introduction allows you to add new methods or attributes to existing classes.                                                                                                                                         |
| Target object | The object being advised by one or more aspects, this object will always be a proxied object. Also referred to as the advised object.                                                                                    |
| Weaving       | Weaving is the process of linking aspects with other application types or objects to create an advised object. This can be done at compile time, load time, or at runtime.                                               |

### Types of Advice

Spring aspects can work with five kinds of advice mentioned below:

| Advice          | Description                                                                          |
|-----------------|--------------------------------------------------------------------------------------|
| before          | Run advice before the method execution.                                              |
| after           | Run advice after the method execution regardless of its outcome.                     |
| after-returning | Run advice after the method execution only if method completes successfully          |
| after-throwing  | Run advice after the method execution only if method exits by throwing an exception. |
| around          | Run advice before and after the advised method is invoked.                           |

| Advice          | Interface            | Overriding Method                                                                                 |
|-----------------|----------------------|---------------------------------------------------------------------------------------------------|
| before          | MethodBeforeAdvice   | before(Method method, Object[] args, Object target) throws Throwable                              |
| after           |                      |                                                                                                   |
| after-returning | AfterReturningAdvice | afterReturning(Object returnValue, Method method, Object[] args, Object target) throws Throwable  |
| after-throwing  | ThrowsAdvice         | afterThrowing(IllegalArgumentException e) throws Throwable                                        |
| around          | MethodInterceptor    | invoke(MethodInvocation methodInvocation) throws Throwable<br>Call methodInvocation.proceed() too |

```
<bean id="customerService" class="com.mesonsoft.customer.services.CustomerService"/>
<bean id="customerServiceInterceptor" class="com.mesonsoft.CustomerServiceInterceptor"/>
<bean id="customerServiceProxy" class="org.springframework.aop.framework.ProxyFactoryBean">
  <property name="target" ref="customerService" />
  <property name="interceptorNames">
    <list>
      <value>customerServiceInterceptor</value>
    </list>
  </property>
</bean>
```

To use Spring proxy, you need to add CGLIB2 library.

```
<dependency>
  <groupId>cglib</groupId>
  <artifactId>cglib</artifactId>
  <version>2.2.2</version>
</dependency>
```

## Custom Aspects Implementation

Spring supports the **@AspectJ annotation style** approach and the **schema-based** approach to implement custom aspects.

| Approach         | Description                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------|
| XML Schema based | Aspects are implemented using regular classes along with XML based configuration as shown above.           |
| @AspectJ based   | @AspectJ refers to a style of declaring aspects as regular Java classes annotated with Java 5 annotations. |

## Bean Factory (Spring Core Module)

A BeanFactory is like a factory class that contains a collection of beans. The BeanFactory holds bean definitions of multiple beans within itself and then instantiates the bean whenever asked for by the clients.

- BeanFactory is able to create associations between collaborating objects as they are instantiated. This removes the burden of configuration from bean itself and the beans client.
- BeanFactory also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

## Application Context (Spring Context Module)

A bean factory is fine to simple applications, but to take advantage of the full power of the Spring framework, you may want to move up to Spring's more advanced container, the Application Context. On the surface, an application context is same as a bean factory. Both load bean definitions, wire beans together, and dispense beans upon request. But it also:

- Provides a means for resolving text messages, including support for internationalization of those messages.
- Provides a generic way to load file resources, such as images.
- Publishes events to beans that are registered as listeners.
- Certain operations on the container or beans in the container can be handled both programmatically and declaratively.

The three commonly used implementation of 'Application Context' are

- **ClassPathXmlApplicationContext** : It loads context definition from an XML file located in the **classpath**, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code.

```
ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
```

- **FileSystemXmlApplicationContext** : It loads context definition from an XML file in the **filesystem**. The application context is loaded from the file system by using the code.

```
ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");
```

- **XmlWebApplicationContext** : It loads context definition from an XML file contained **within a web application**.

## Differences between Bean Factory and Application Context

On the surface, an application context is same as a bean factory. But application context offers much more.

- Application contexts provide a means for resolving text messages, including support for i18n of those messages.

- Application contexts provide a generic way to load file resources, such as images.
- Application contexts can publish events to beans that are registered as listeners.
- Certain operations on the container or beans in the container, which have to be handled in a programmatic fashion with a bean factory, can be handled declaratively in an application context.
- ResourceLoader support: Spring's Resource interface is a flexible generic abstraction for handling low-level resources. An application context itself is a ResourceLoader, Hence provides an application with access to deployment-specific Resource instances.
- MessageSource support: The application context implements MessageSource, an interface used to obtain localized messages, with the actual implementation being pluggable

## Spring JDBC Template

Spring JDBC provides several approaches and correspondingly different classes to interface with the database. I'm going to take classic and the most popular approach which makes use of **JdbcTemplate** class of the framework. This is the central framework class that manages all the database communication and exception handling.

### JdbcTemplate Class

The JdbcTemplate class executes SQL queries, updates statements and stored procedure calls, performs iteration over resultsets and extracts the returned parameter values. It also catches JDBC exceptions and translates them to the generic, more informative, exception hierarchy defined in the org.springframework.dao package.

Instances of the JdbcTemplate class are **threadsafe** once configured. So you can configure a single instance of a JdbcTemplate and then safely inject this shared reference into multiple DAOs.

A common practice when using the JdbcTemplate class is to configure a *DataSource* in your Spring configuration file, and then dependency-inject that shared DataSource bean into your DAO classes, and the JdbcTemplate is created in the setter for the DataSource.

### Configuring Data Source

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
  <property name="url" value="jdbc:mysql://localhost:3306/TEST"/>
  <property name="username" value="root"/>
  <property name="password" value="password"/>
</bean>
```

### Data Access Object (DAO)

DAO stands for data access object which is commonly used for database interaction. DAOs exist to provide a means to read and write data to the database and they should expose this functionality through an interface by which the rest of the application will access them.

The Data Access Object (DAO) support in Spring makes it easy to work with data access technologies like JDBC, Hibernate, JPA or JDO in a consistent way.

### Executing SQL statements

Let us see how we can perform CRUD (Create, Read, Update and Delete) operations on database tables using SQL and jdbcTemplate object.

#### Querying for an integer:

```
String SQL = "select count(*) from Student";
int rowCount = jdbcTemplateObject.queryForInt( SQL );
```

#### Querying for a long:

```
String SQL = "select count(*) from Student";
long rowCount = jdbcTemplateObject.queryForLong( SQL );
```

#### A simple query using a bind variable:

```
String SQL = "select age from Student where id = ?";
int age = jdbcTemplateObject.queryForInt(SQL, new Object[]{10});
```

#### Querying for a String:

```
String SQL = "select name from Student where id = ?";
String name = jdbcTemplateObject.queryForObject(SQL, new Object[]{10}, String.class);
```

#### Querying and returning an object:

```
String SQL = "select * from Student where id = ?";
Student student = jdbcTemplateObject.queryForObject(SQL,
    new Object[]{10}, new StudentMapper());

public class StudentMapper implements RowMapper<Student> {
    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
        Student student = new Student();
        student.setID(rs.getInt("id"));
        student.setName(rs.getString("name"));
        student.setAge(rs.getInt("age"));
        return student;
    }
}
```

### Querying and returning multiple objects:

```
String SQL = "select * from Student";
List<Student> students = jdbcTemplateObject.query(SQL,
    new StudentMapper());

public class StudentMapper implements RowMapper<Student> {
    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
        Student student = new Student();
        student.setID(rs.getInt("id"));
        student.setName(rs.getString("name"));
        student.setAge(rs.getInt("age"));
        return student;
    }
}
```

### Inserting a row into the table:

```
String SQL = "insert into Student (name, age) values (?, ?)";
jdbcTemplateObject.update( SQL, new Object[]{"Zara", 11});
```

### Updating a row into the table:

```
String SQL = "update Student set name = ? where id = ?";
jdbcTemplateObject.update( SQL, new Object[]{"Zara", 10});
```

### Deleting a row from the table:

```
String SQL = "delete Student where id = ?";
jdbcTemplateObject.update( SQL, new Object[]{20});
```

### Executing DDL Statements

You can use the **execute(..)** method from *jdbcTemplate* to execute any SQL statements or DDL statements. Following is an example to use CREATE statement to create a table:

```
String SQL = "CREATE TABLE Student( " +
    "ID INT NOT NULL AUTO_INCREMENT, " +
    "NAME VARCHAR(20) NOT NULL, " +
    "AGE INT NOT NULL, " +
    "PRIMARY KEY (ID));"

jdbcTemplateObject.execute( SQL );
```

## Context Configuration Annotations

These annotations are used by Spring to guide creation and injection of beans.

| Annotation               | Use                                     | Description                                                                                                                                                             |
|--------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>@Autowired</b>        | Constructor, Field, Method              | Declares a constructor, field, setter method, or configuration method to be autowired by type. Items annotated with @Autowired do not have to be public.                |
| <b>@ConfigurableType</b> |                                         | Used with to declare types whose properties should be injected, even if they are not instantiated by Spring. Typically used to inject the properties of domain objects. |
| <b>@Order</b>            | Type, Method, Field                     | Defines ordering, as an alternative to implementing the org.springframework.core.Ordered interface.                                                                     |
| <b>@Qualifier</b>        | Field, Parameter, Type, Annotation Type | Guides autowiring to be performed by means other than by type.                                                                                                          |
| <b>@Required</b>         | Method (setters)                        | Specifies that a particular property must be injected or else the configuration will fail.                                                                              |
| <b>@Scope</b>            | Type                                    | Specifies the scope of a bean, singleton, prototype, request, session, or some custom scope.                                                                            |

### Autowiring without Setter Methods

The XML Configuration file should have **<context:annotation-config>** tag to activate the annotation based configuration. @Autowired can be used on any method (not just setter methods). The wiring can be done through any method, as illustrated here:

```
@Configurable(autowire = Autowire.BY_TYPE)
public class Treasuse {
    @Autowired
    public void directionsToTreasure(TreasureMap treasureMap) {
        this.treasureMap = treasureMap;
    }
}
```

And even on member variables:

```
@Autowired
private TreasureMap treasureMap;
```

To resolve any autowiring ambiguity, use the @Qualifier attribute with @Autowired.

```
@Autowired
@Qualifier("mapToTortuga")
private TreasureMap treasureMap;
```

## Ensuring that Required Properties are Set

To ensure that a property is injected with a value, use the `@Required` annotation:

```
@Required
public void setTreasureMap(TreasureMap treasureMap) {
    this.treasureMap = treasureMap;
}
```

In this case, the "treasureMap" property must be injected or else Spring will throw a `BeanInitializationException` and context creation will fail.

## Stereotyping Annotations

These annotations are used to stereotype classes with regard to the application tier that they belong to. Classes that are annotated with one of these annotations will automatically be registered in the Spring application context if `<context:component-scan>` is in the Spring XML configuration. It is required to activate the Spring MVC annotation capability.

In addition, if a `PersistenceExceptionTranslationPostProcessor` is configured in Spring, any bean annotated with `@Repository` will have `SQLExceptions` thrown from its methods translated into one of Spring's unchecked `DataAccessExceptions`.

| Annotation               | Use  | Description                                                                                                                                                                                                                       |
|--------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@Component</code>  | Type | <b>Generic</b> stereotype annotation for any Spring-managed component.                                                                                                                                                            |
| <code>@Controller</code> | Type | Stereotypes a component as a Spring MVC controller in the <b>presentation layer</b> .                                                                                                                                             |
| <code>@Repository</code> | Type | Stereotypes a component as a repository in the <b>persistence layer</b> . Also indicates that <code>SQLExceptions</code> thrown from the component's methods should be translated into Spring <code>DataAccessExceptions</code> . |
| <code>@Service</code>    | Type | Stereotypes a component as a service in the <b>service layer</b> .                                                                                                                                                                |

## Automatically Configuring Beans

In the previous section, you saw how to automatically wire a bean's properties using the `@Autowired` annotation. But it is possible to take autowiring to a new level by automatically registering beans in Spring. To get started with automatic registration of beans, first annotate the bean with one of the stereotype annotations, such as `@Component`:

```
@Component
public class Pirate {
    private TreasureMap treasureMap;

    @Autowired
    public void setTreasureMap(TreasureMap treasureMap) {
        this.treasureMap = treasureMap;
    }
}
```

You can specify a name for the bean by passing it as the value of `@Component`.

```
@Component("jackSparrow")  
public class Pirate { ... }
```

#### Specifying Scope For Auto-Configured Beans

By default, all beans in Spring, including auto-configured beans, are scoped as singleton. But you can specify the scope using the `@Scope` annotation. For example:

```
@Component  
@Scope("prototype")  
public class Pirate { ... }
```

This specifies that the pirate bean be scoped as a prototype bean.

#### Creating Custom Stereotypes

Autoregistering beans are a great way to cut back on the amount of XML required to configure Spring. But it may bother you that your autoregistered classes are annotated with Spring-specific annotations. If you're looking for a more non-intrusive way to autoregister beans, you have two options:

Create your own custom stereotype annotation. Doing so is as simple as creating a custom annotation that is itself annotated with `@Component`:

```
@Component  
public interface MyComponent {  
}  
}
```

Or add a filter to `<context:component-scan>` to scan for annotations that it normally would not:

```
<context:component-scan base-package="com.habuma.pirates">
```

```
<context:include-filter type="annotation" expression="com.habuma.MyComponent" />  
<context:exclude-filter type="annotation" expression="org.springframework.stereotype.Component"  
/>
```

```
</context:component-scan>
```

In this case, the `@MyComponent` custom annotation has been added to the list of annotations that are scanned for, but `@Component` has been excluded (that is, `@Component` annotated classes will no longer be autoregistered).

Regardless of which option you choose, you should be able to autoregister beans by annotating their classes with the custom annotation:

```
@MyComponent  
public class Pirate { ... }
```

## Spring MVC Annotations

These annotations were introduced in Spring 2.5 to make it easier to create Spring MVC applications with minimal XML configuration and without extending one of the many implementations of the Controller interface.

| Annotation         | Use               | Description                                                                                                                                                         |
|--------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @Controller        | Type              | Stereotypes a component as a Spring MVC controller.                                                                                                                 |
| @InitBinder        | Method            | Annotates a method that customizes data binding.                                                                                                                    |
| @ModelAttribute    | Method, Parameter | When applied to a method, used to preload the model with the value returned from the method. When applied to a parameter, binds a model attribute to the parameter. |
| @RequestMapping    | Method, Type      | Maps a URL pattern and/or HTTP method to a method or controller type.                                                                                               |
| @RequestParam      | Parameter         | Binds a request parameter to a method parameter.                                                                                                                    |
| @SessionAttributes | Type              | Specifies that a model attribute should be stored in the session.                                                                                                   |

## Setting up Spring for Annotated Controllers

Before we can use annotations on Spring MVC controllers, we'll need to add a few lines of XML to tell Spring that our controllers will be annotation-driven. First, so that we won't have to register each of our controllers individually as <bean>s, we'll need a <context:component-scan>:

```
<context:component-scan base-package= "com.mesonsoft"/>
```

In addition to autoregistering @Component-annotated beans, <context:component-scan> also autoregisters beans that are annotated with @Controller. We'll see a few examples of @Controller-annotated classes in a moment.

But first, we'll also need to tell Spring to honor the other Spring MVC annotations. For that we'll need <context:annotation-config> : <context:annotation-config/>

## Transaction Annotations

The @Transactional annotation is used along with the <tx:annotation-driven> element to declare transactional boundaries and rules as class and method metadata in Java.

| Annotation     | Use          | Description                                                               |
|----------------|--------------|---------------------------------------------------------------------------|
| @Transactional | Method, Type | Declares transactional boundaries and rules on a bean and/or its methods. |

## Annotating Transactional Boundaries

To use Spring's support for annotation-declared transactions, you'll first need to add a small amount of XML to the Spring configuration:

```
<beans><tx:annotation-driven /></beans>
```

The <tx:annotation-driven> element tells Spring to keep an eye out for beans that are annotated with @Transactional. In addition, you'll also need a platform transaction manager bean declared in the Spring context. For example, if your application uses Hibernate, you'll want to include the

### HibernateTransactionManager:

```
<bean id="transactionManager" class="org.springframework.orm.hibernate3.  
HibernateTransactionManager"><property name="sessionFactory" ref="sessionFactory"/></bean>
```

With the basic plumbing in place, you're ready to start annotating the transactional boundaries:

```
@Transactional(propagation=Propagation.SUPPORTS, readOnly=true)  
public class TreasureRepositoryImpl implements TreasureRepository {  
    @Transactional(propagation=Propagation.REQUIRED, readOnly=false)  
    public void storeTreasure(Treasure treasure) {...}  
}
```

At the class level, `@Transactional` is declaring that all methods should support transactions and be read-only. But, at the method-level, `@Transactional` declares that the `storeTreasure()` method requires a transaction and is not read-only.

### Aspect Annotations

For defining aspects, Spring leverages the set of annotations provided by AspectJ.

| Annotation                         | Use    | Description                                                                                                             |
|------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------|
| <code>@Aspect</code>               | Type   | Declares a class to be an aspect.                                                                                       |
| <code>@After</code>                | Method | Declares a method to be called after a pointcut completes.                                                              |
| <code>@AfterReturning</code>       | Method | Declares a method to be called after a pointcut returns successfully.                                                   |
| <code>@AfterThrowing</code>        | Method | Declares a method to be called after a pointcut throws an exception.                                                    |
| <code>@Around</code>               | Method | Declares a method that will wrap the pointcut.                                                                          |
| <code>@Before</code>               | Method | Declares a method to be called before proceeding to the pointcut.                                                       |
| <code>@DeclareParentsStatic</code> | Field  | Declares that matching types should be given new parents, that is, it introduces new functionality into matching types. |
| <code>@Pointcut</code>             | Method | Declares an empty method as a pointcut placeholder method.                                                              |

What's important to note, however, is that while you can use AspectJ annotations to define Spring aspects, those aspects will be defined in the context of Spring AOP and will not be handled by the AspectJ runtime. This is significant because Spring AOP is limited to proxying method invocations and does not provide for the more exotic pointcuts (constructor interception, field interception, etc.) offered by AspectJ.

### Annotating Aspects

To use AspectJ annotations to create Spring aspects, you'll first need to provide a bit of Spring XML plumbing:

```
<beans><aop:aspectj-autoproxy/></beans>
```

The <aop:aspectj-autoproxy> element tells Spring to watch for beans annotated with AspectJ annotations and, if it finds any, to use them to create aspects. Then you can annotate bean classes to be aspects:

```
@Aspect
public class ChantySinger {

    @Pointcut("execution(* Pirate.plunder(..)")
    public void plunderPC() {}

    @Before("plunderPC()")
    public void singYoHo() {
        ...
    }

    @AfterReturning("plunderPC()")
    public void singAPiratesLifeForMe() {
        ...
    }
}
```

This simple annotation-based aspect has a pointcut that is triggered by the execution of a plunder() method on the Pirate class. Before the Pirate.plunder() method is executed, the singYoHo() method is called. Then, after the Pirate.plunder() method returns successfully, the singAPiratesLifeForMe() method is invoked.

Note the rather odd looking plunderPC() method. It is annotated with @Pointcut to indicate that this method is a pointcut placeholder. The key thing here is that the most interesting stuff happens in the annotation itself and not in the method. In fact, pointcut placeholder methods must be empty methods and return void.

## JSR-250 Annotations

In addition to Spring's own set of annotations, Spring also supports a few of the annotations defined by JSR-250, which is the basis for the annotations used in EJB 3.

| Annotation           | Use           | Description                                                                                                                                             |
|----------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| @PostConstructMethod |               | Indicates a method to be invoked after a bean has been created and dependency injection is complete. Used to perform any initialization work necessary. |
| @PreDestroy          | Method        | Indicates a method to be invoked just before a bean is removed from the Spring context. Used to perform any cleanup work necessary.                     |
| @Resource            | Method, Field | Indicates that a method or field should be injected with a named resource (by default, another bean).                                                   |

## Wiring Bean Properties with @Resource

Using @Resource, you can wire a bean property by name:

```
public class Pirate {  
    @Resource  
    private TreasureMap treasureMap;  
}
```

In this case, Spring will attempt to wire the "treasureMap" property with a reference to a bean whose ID is "treasureMap". If you'd rather explicitly choose another bean to wire into the property, specify it to the name attribute:

```
public class Pirate {  
    @Resource(name="mapToSkullIsland")  
    private TreasureMap treasureMap;  
}
```

## Initialization and Destruction Methods

Using JSR-250's @PostConstruct and @PreDestroy methods, you can declare methods that hook into a bean's lifecycle. For example, consider the following methods added to the Pirate class:

```
public class Pirate {  
    @PostConstruct  
    public void wakeUp() { ... }  
    @PreDestroy  
    public void goAway() { ... }  
}
```

As annotated, the wakeUp() method will be invoked just after Spring instantiates the bean and goAway() will be invoked just before the bean is removed from the Spring container.

## Writing a Spring-Aware JUnit Test

The key to writing a Spring-aware test is to annotate the test class with @RunWith, specifying **SpringJUnit4ClassRunner** as the class runner behind the test:

```
@RunWith(SpringJUnit4ClassRunner.class)  
public class PirateTest {  
    ...  
}
```

In this case, the Spring test runner will try to load a Spring application context from a file named `PirateTest-context.xml`. If you'd rather specify one or more XML files to load the application context from, you can do that with `@ContextConfiguration`:

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(locations = { "pirates.xml" })
public class PirateTest {

    @Autowired
    private Pirate pirate;

    @Autowired
    private TreasureMap treasureMap;

    @Test
    public void annotatedPropertyShouldBeAutowired() {
        assertNotNull(treasureMap);
        assertEquals(treasureMap, pirate.getTreasureMap());
    }
}
```

### Accessing the Spring Context in a Test

If you need the Spring application context itself in a test, you can autowire it into the test the same as if it were a bean in the context:

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(locations = { "pirates.xml" })
public class PirateTest {

    @Autowired
    private Pirate pirate;

    @Autowired
    private ApplicationContext applicationContext;

    @Test
    public void annotatedPropertyShouldBeAutowired() {
        assertEquals(applicationContext.getBean("treasureMap"), pirate
            .getTreasureMap());
    }
}
```

## Spring Transaction Management

A database transaction is a sequence of actions that are treated as a single unit of work. These actions should either complete entirely or take no effect at all. Transaction management is an important part of RDBMS oriented enterprise applications to ensure data integrity and consistency. The concept of transactions can be described with following four key properties described as **ACID**:

**Atomicity:** A transaction should be treated as a single unit of operation which means either the entire sequence of operations is successful or unsuccessful.

**Consistency:** This represents the consistency of the referential integrity of the database, unique primary keys in tables etc.

**Isolation:** There may be many transactions processing with the same data set at the same time, each transaction should be isolated from others to prevent data corruption.

**Durability:** Once a transaction has completed, the results of this transaction have to be made permanent and cannot be erased from the database due to system failure.

A real RDBMS database system will guarantee all the four properties for each transaction. The simplistic view of a transaction issued to the database using SQL is as follows:

- Begin the transaction using *begin transaction* command.
- Perform various deleted, update or insert operations using SQL queries.
- If all the operation are successful then perform *commit* otherwise *rollback* all the operations.

Spring framework provides an abstract layer on top of different underlying transaction management APIs. The Spring's transaction support aims to provide an alternative to EJB transactions by adding transaction capabilities to POJOs. Spring supports both programmatic and declarative transaction management. EJBs requires an application server, but Spring transaction management can be implemented without a need of application server.

## Local versus Global Transactions

Local transactions are specific to a **single transactional resource** like a JDBC connection, whereas global transactions can span **multiple transactional resources** like transaction in a distributed system.

Local transaction management can be useful in a centralized computing environment where application components and resources are located at a single site, and transaction management only involves a local data manager running on a single machine. Local transactions are easier to be implemented.

Global transaction management is required in a distributed computing environment where all the resources are distributed across multiple systems. In such a case transaction management needs to be done both at local and global levels. A distributed or a global transaction is executed across multiple systems, and its execution requires coordination between the global transaction management system and all the local data managers of all the involved systems.

## Programmatic versus Declarative

Spring supports two types of transaction management:

**Programmatic Transaction Management:** This means that you have to manage the transaction with the help of programming. That gives you extreme flexibility, but it is difficult to maintain.

**Declarative Transaction Management:** This means you separate transaction management from the business code. You only use annotations or XML based configuration to manage the transactions.

Declarative transaction management is preferable over programmatic transaction management though it is less flexible than programmatic transaction management, which allows you to control transactions through your code. But as a kind of crosscutting concern, declarative transaction management can be modularized with the AOP approach. Spring supports declarative transaction management through the Spring AOP framework.

## Spring Transaction Abstractions

The key to the Spring transaction abstraction is defined by **PlatformTransactionManager** interface.

```
public interface PlatformTransactionManager {  
    TransactionStatus getTransaction(TransactionDefinition definition);  
    throws TransactionException;  
    void commit(TransactionStatus status) throws TransactionException;  
    void rollback(TransactionStatus status) throws TransactionException;  
}
```

| S.N. | Method & Description                                                                                                                                                                                  |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>TransactionStatus getTransaction(TransactionDefinition definition)</b><br>This method returns a currently active transaction or create a new one, according to the specified propagation behavior. |
| 2    | <b>void commit(TransactionStatus status)</b><br>This method commits the given transaction, with regard to its status.                                                                                 |
| 3    | <b>void rollback(TransactionStatus status)</b><br>This method performs a rollback of the given transaction.                                                                                           |

## TransactionDefinition

The **TransactionDefinition** is the core interface of the transaction support in Spring and it is defined as below:

```
public interface TransactionDefinition {  
    int getPropagationBehavior();  
    int getIsolationLevel();  
    String getName();  
    int getTimeout();  
    boolean isReadOnly();  
}
```

| S.N. | Method & Description                                                                                                                                                 |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>int getPropagationBehavior()</b><br>This method returns the propagation behavior. Spring offers the entire transaction propagation options familiar from EJB CMT. |
| 2    | <b>int getIsolationLevel()</b><br>This method returns the degree to which this transaction is isolated from the work of other transactions.                          |
| 3    | <b>String getName()</b><br>This method returns the name of this transaction.                                                                                         |
| 4    | <b>int getTimeout()</b><br>This method returns the time in seconds in which the transaction must complete.                                                           |
| 5    | <b>boolean isReadOnly()</b><br>This method returns whether the transaction is read-only.                                                                             |

## Isolation Level

| S.N. | Isolation & Description                                                                                                                               |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>TransactionDefinition.ISOLATION_DEFAULT</b><br>This is the default isolation level.                                                                |
| 2    | <b>TransactionDefinition.ISOLATION_READ_UNCOMMITTED</b><br>Indicates that dirty reads, non-repeatable reads and phantom reads can occur.              |
| 3    | <b>TransactionDefinition.ISOLATION_READ_COMMITTED</b><br>Indicates that dirty reads are prevented; non-repeatable reads and phantom reads can occur.  |
| 4    | <b>TransactionDefinition.ISOLATION_REPEATABLE_READ</b><br>Indicates that dirty reads and non-repeatable reads are prevented; phantom reads can occur. |
| 5    | <b>TransactionDefinition.ISOLATION_SERIALIZABLE</b><br>Indicates that dirty reads, non-repeatable reads and phantom reads are prevented.              |

## Propagation Behavior

| S.N. | Propagation & Description                                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>TransactionDefinition.PROPGATION_MANDATORY</b><br>Support a current transaction; throw an exception if no current transaction exists.             |
| 2    | <b>TransactionDefinition.PROPGATION_NESTED</b><br>Execute within a nested transaction if a current transaction exists.                               |
| 3    | <b>TransactionDefinition.PROPGATION_NEVER</b><br>Do not support a current transaction; throw an exception if a current transaction exists.           |
| 4    | <b>TransactionDefinition.PROPGATION_NOT_SUPPORTED</b><br>Do not support a current transaction; rather always execute non-transactionally.            |
| 5    | <b>TransactionDefinition.PROPGATION_REQUIRED</b><br>Support a current transaction; create a new one if none exists.                                  |
| 6    | <b>TransactionDefinition.PROPGATION_REQUIRES_NEW</b><br>Create a new transaction, suspending the current transaction if one exists.                  |
| 7    | <b>TransactionDefinition.PROPGATION_SUPPORTS</b><br>Support a current transaction; execute non-transactionally if none exists.                       |
| 8    | <b>TransactionDefinition.TIMEOUT_DEFAULT</b><br>Use the default timeout of the underlying transaction system, or none if timeouts are not supported. |

The **TransactionStatus** interface provides a simple way for transactional code to control transaction execution and query transaction status.

```
public interface TransactionStatus extends SavepointManager {  
    boolean isNewTransaction();  
    boolean hasSavepoint();  
    void setRollbackOnly();  
    boolean isRollbackOnly();  
    boolean isCompleted();  
}
```

| S.N. | Method & Description                                                                                                         |
|------|------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>boolean hasSavepoint()</b><br>This method returns whether this transaction internally carries a savepoint.                |
| 2    | <b>boolean isCompleted()</b><br>This method returns whether this transaction is completed (either committed or rolled back). |
| 3    | <b>boolean isNewTransaction()</b><br>This method returns true in case the present transaction is new.                        |
| 4    | <b>boolean isRollbackOnly()</b><br>This method returns whether the transaction has been marked as rollback-only.             |
| 5    | <b>void setRollbackOnly()</b><br>This method sets the transaction rollback-only.                                             |

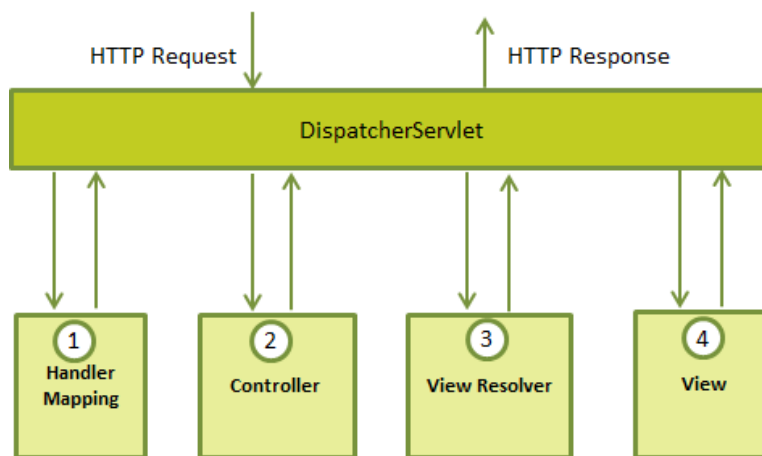
## Spring MVC Framework

The Spring web MVC framework provides **model-view-controller architecture** and ready components that can be used to develop **flexible and loosely coupled web applications**. The MVC pattern results in separating the different aspects of the application (input logic, business logic, and UI logic), while providing a loose coupling between these elements.

- The **Model** encapsulates the application data and in general they will consist of POJO.
- The **View** is responsible for rendering the model data and in general it generates HTML output that the client's browser can interpret.
- The **Controller** is responsible for processing user requests and building appropriate model and passes it to the view for rendering.

### The DispatcherServlet

The Spring Web model-view-controller (MVC) framework is designed around a *DispatcherServlet* that handles all the HTTP requests and responses. The request processing workflow of the Spring Web MVC *DispatcherServlet* is illustrated in the following diagram:



Following is the sequence of events corresponding to an incoming HTTP request to *DispatcherServlet*:

1. After receiving an HTTP request, *DispatcherServlet* consults the *HandlerMapping* to call the appropriate *Controller*.
2. The *Controller* takes the request and calls the appropriate service methods based on used GET or POST method. The service method will set model data based on defined business logic and returns view name to the *DispatcherServlet*.
3. The *DispatcherServlet* will take help from *ViewResolver* to pick up the defined view for the request.
4. Once view is finalized, The *DispatcherServlet* passes the model data to the view which is finally rendered on the browser.
5. All the above mentioned components ie. *HandlerMapping*, *Controller* and *ViewResolver* are parts of **WebApplicationContext** which is an extension of the plain *ApplicationContext* with some extra features necessary for web applications.

## Spring MVC Configuration

You need to map requests that you want the *DispatcherServlet* to handle, by using a URL mapping in the **web.xml** file. The **web.xml** file will be kept *WebContent/WEB-INF* directory of your web application. OK, upon initialization of **HelloWeb DispatcherServlet**, the framework will try to load the application context from a file named **[servlet-name]-servlet.xml** located in the application's *WebContent/WEB-INF* directory. In this case our file will be **HelloWeb-servlet.xml**.

Next, `<servlet-mapping>` tag indicates what URLs will be handled by the which *DispatcherServlet*. Here all the HTTP requests ending with **.jsp** will be handled by the **HelloWeb DispatcherServlet**.

If you do not want to go with default filename as *[servlet-name]-servlet.xml* and default location as *WebContent/WEB-INF*, you can customize this file name and location by adding the servlet listener *ContextLoaderListener* in your **web.xml** file as follows:

```
<web-app>
  <servlet>
    <servlet-name>HelloWeb</servlet-name>
    <servlet-class>
      org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/HelloWeb-servlet.xml</param-value>
  </context-param>
  <listener>
    <listener-class>
      org.springframework.web.context.ContextLoaderListener
    </listener-class>
  </listener>
</web-app>
```

Now, let us check the required configuration for **HelloWeb-servlet.xml** file, placed in your web application's *WebContent/WEB-INF* directory:

```
<context:component-scan base-package="com.mesonsoft" />

<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/jsp/" />
  <property name="suffix" value=".jsp" />
</bean>
```

Following are the important points about **HelloWeb-servlet.xml** file:

- The *[servlet-name]-servlet.xml* file will be used to create the beans defined, overriding the definitions of any beans defined with the same name in the global scope.
- The `<context:component-scan...>` tag will be used to **activate Spring MVC annotation scanning capability** which allows to make use of annotations like **@Controller** and **@RequestMapping** etc.
- The **InternalResourceViewResolver** will have rules defined to resolve the view names. As per the above defined rule, a logical view named **hello** is delegated to a view implementation located at */WEB-INF/jsp/hello.jsp*.

## Defining a Controller

DispatcherServlet delegates the request to the controllers to execute the functionality specific to it. The **@Controller** annotation indicates that a particular class serves the role of a controller. The **@RequestMapping** annotation is used to map a URL to either an entire class or a particular handler method.

```
@Controller
@RequestMapping("/hello")
public class HelloController{
    @RequestMapping(method = RequestMethod.GET)
    public String printHello(ModelMap model) {
        model.addAttribute("message", "Hello Spring MVC Framework!");
        return "hello";
    }
}
```

The **@Controller** annotation defines the class as a Spring MVC controller. Here, the first usage of **@RequestMapping** indicates that all handling methods on this controller are relative to the **/hello** path. Next annotation **@RequestMapping(method=RequestMethod.GET)** is used to declare the *printHello()* method as the controller's default service method to handle HTTP GET request. You can define another method to handle any POST request at the same URL.

You can write above controller in another form where you can add additional attributes in *@RequestMapping* as follows:

```
@Controller
public class HelloController{

    @RequestMapping(value = "/hello", method = RequestMethod.GET)
    public String printHello(ModelMap model) {
        model.addAttribute("message", "Hello Spring MVC Framework!");
        return "hello";
    }
}
```

## Spring MVC Handler Interceptor

Spring MVC allows you to intercept the web request and response through handler interceptors. The handler interceptor has to implement the *HandlerInterceptor* interface, which contains three methods:

1. **preHandle()** – Called before the handler execution. It returns a Boolean value. True means continue the handler execution chain; otherwise, stop the execution chain and return it.
2. **postHandle()** – Called after the handler execution. It allows you to manipulate the *ModelAndView* object before render it to the view page.
3. **afterCompletion()** – Called after the user request has been processed. Seldom use, we can't find any use case.

It's recommended to extend the *HandlerInterceptorAdapter* class for the convenient default implementations.

# Chapter 24 – Hibernate

## *ORM (Object/Relational Mapping)*

---

ORM stands for Object Relational Mapping. ORM is the automated persistence of objects in a Java application to the tables in a relational database.

An ORM solution consists of the following four pieces:

- API for performing basic CRUD operations
- API to express queries referring to classes
- Facilities to specify metadata
- Optimization facilities such as dirty checking, lazy associations fetching and so on

The ORM levels are:

- Pure relational (stored procedure)
- Light object mapping (JDBC)
- Medium object mapping
- Full object Mapping (composition, inheritance, polymorphism, persistence by reachability)

### **Hibernate Framework**

Hibernate is a pure Java object relational mapping (ORM) and persistence framework that allows you to map plain old java objects to relational database tables using (XML) configuration files.



The main advantage of ORM like hibernate is that it shields developers from messy SQL. Apart from this, ORM provides following benefits:

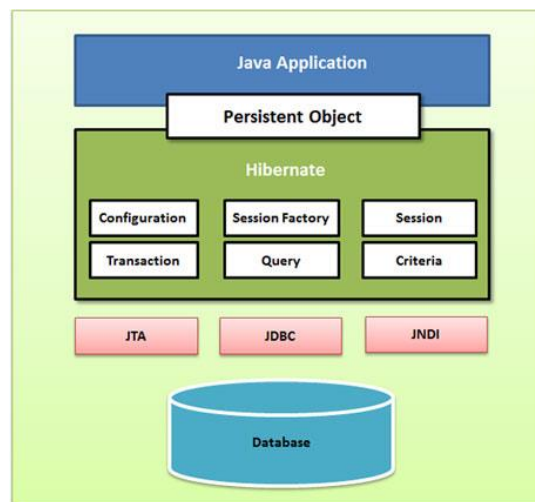
- **Improved productivity**
  - High-level object-oriented API
  - Less Java code to write
  - No SQL to write
- **Improved performance**
  - Sophisticated caching
  - Lazy loading
  - Eager loading
- **Improved maintainability**
  - A lot less code to write and maintain
- **Improved portability**
  - ORM framework generates database-specific SQL for you

**Hibernate simplifies:**

- Saving and retrieving your domain objects
- Making database column and table name changes
- Centralizing pre save and post retrieve logic
- Complex joins for retrieving related items
- Schema creation from object model

**Core Interfaces of Hibernate Framework:**

1. Session Interface
2. SessionFactory Interface
3. Configuration Interface
4. Transaction Interface
5. Query and Criteria Interface



## General Flow:

The general flow of Hibernate communication with RDBMS is:

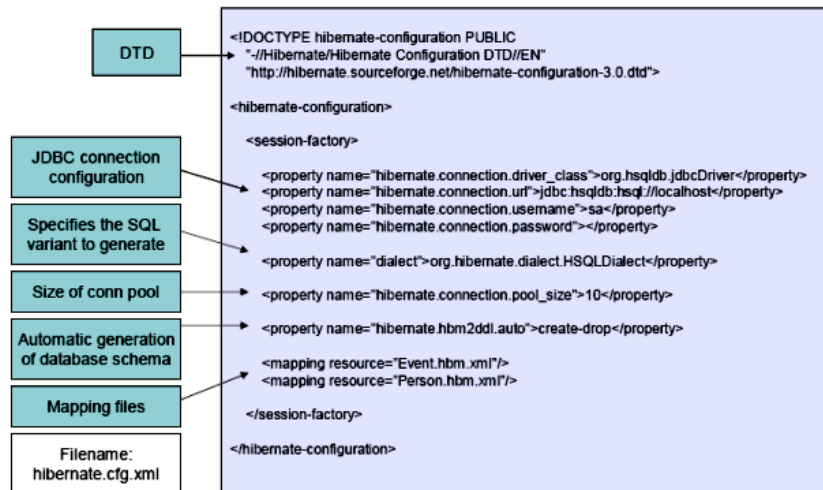
1. Load the Hibernate configuration file and create configuration object. It will automatically load all hbm mapping files
2. Create session factory from configuration object
3. Get one session from this session factory
4. Begin the transaction, and create HQL query
5. Execute the query to get a list containing Java objects
6. Commit or Rollback the transaction based on the query execution outcome

## Configuration Interface

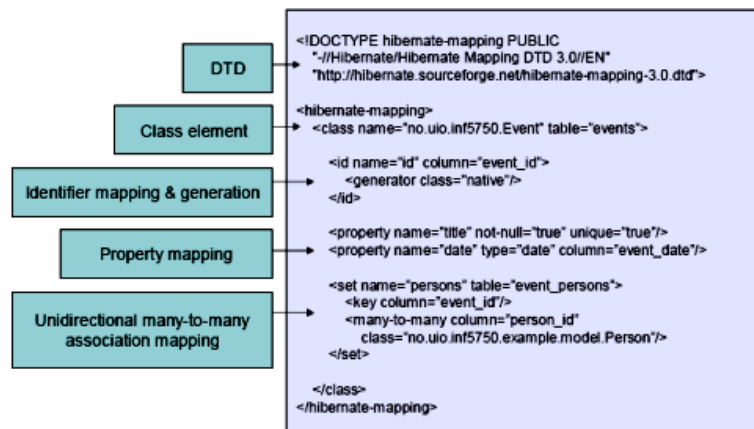
The most common methods of Hibernate configuration are:

- Programmatic configuration
- Declarative configuration (hibernate.cfg.xml)

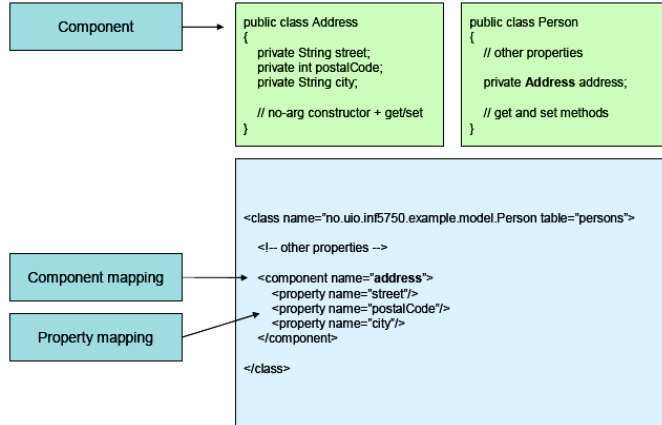
Following are the important tags of the Hibernate configuration file (hibernate.cfg.xml):



Hibernate mapping file tells Hibernate which tables and columns to use to load and store objects. Some of the important tags of a typical mapping file (person.hbm.xml) are given below:



## Component Mapping in Hibernate



## Derived Properties

The properties that are not mapped to a column, but calculated at runtime by evaluation of an expression are called derived properties. The expression can be defined using the **formula** attribute of the element.

```
<property name="_LastRevisionDate"
  formula="(select MAX(rev.start_date) from trevision rev where rev.id_detectable = id_detectable and rev.status != 'DRAFT')"
```

type="date" />

## Hibernate DataSource Connection Properties:

Following is the list of important properties you would require to configure for the databases in a **standalone** situation:

| S.N. | Properties and Description                                                                                                   |
|------|------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>hibernate.dialect</b><br>This property makes Hibernate generate the appropriate SQL for the chosen database.              |
| 2    | <b>hibernate.connection.driver_class</b><br>The JDBC driver class.                                                           |
| 3    | <b>hibernate.connection.url</b><br>The JDBC URL to the database instance.                                                    |
| 4    | <b>hibernate.connection.username</b><br>The database username.                                                               |
| 5    | <b>hibernate.connection.password</b><br>The database password.                                                               |
| 6    | <b>hibernate.connection.pool_size</b><br>Limits the number of connections waiting in the Hibernate database connection pool. |
| 7    | <b>hibernate.connection.autocommit</b><br>Allows autocommit mode to be used for the JDBC connection.                         |

If you are using a database along with an **application server and JNDI** then you would have to configure the following properties:

| S.N. | Properties and Description                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>hibernate.connection.datasource</b><br>The JNDI name defined in the application server context you are using for the application. |
| 2    | <b>hibernate.jndi.class</b><br>The InitialContext class for JNDI.                                                                    |
| 3    | <b>hibernate.jndi.&lt;JNDIpropertyname&gt;</b><br>Passes any JNDI property you like to the JNDI <i>InitialContext</i> .              |
| 4    | <b>hibernate.jndi.url</b><br>Provides the URL for JNDI.                                                                              |
| 5    | <b>hibernate.connection.username</b><br>The database username.                                                                       |
| 6    | <b>hibernate.connection.password</b><br>The database password.                                                                       |

### Hibernate Mapping Configuration

```
<hibernate-mapping
    schema="databaseSchemaName"
    catalog="databaseCatalogName"
    default-cascade="none|cascade_style"
    default-access="field|property|ClassName"
    default-lazy="true|false"
    auto-import="true|false"
    package="package.name"
/>
```

- 1 schema (optional): the name of a database schema.
- 2 catalog (optional): the name of a database catalog.
- 3 default-cascade (optional - defaults to none): a default cascade style.
- 4 default-access (optional - defaults to property): the strategy Hibernate should use for accessing all properties. It can be a custom implementation of PropertyAccessor. By mapping the property with default-access="field" in Hibernate metadata. These forces hibernate to bypass the setter method and access the instance variable directly while initializing a newly loaded object.
- 5 default-lazy (optional - defaults to true): the default value for unspecified lazy attributes of class and collection mappings.
- 6 auto-import (optional - defaults to true): specifies whether we can use unqualified class names of classes in this mapping in the query language.
- 7 package (optional): specifies a package prefix to use for unqualified class names in the mapping document.

If you have two persistent classes with the same unqualified name, you should set `auto-import="false"`. An exception will result if you attempt to assign two classes to the same "imported" name.

The `hibernate-mapping` element allows you to nest several persistent `<class>` mappings, as shown above. It is, however, good practice (and expected by some tools) to map only a single persistent class, or a single class hierarchy, in one mapping file and name it after the persistent superclass. For example, `Cat.hbm.xml`, `Dog.hbm.xml`, or if using inheritance, `Animal.hbm.xml`.

## Class Configuration

You can declare a persistent class using the `class` element. For example:

```
<class
  name="ClassName"
  table="tableName"
  discriminator-value="className"
  mutable="true|false"
  schema="databaseSchemaName"
  catalog="databaseCatalogName"
  proxy="ProxyInterface"
  dynamic-update="true|false"
  dynamic-insert="true|false"
  select-before-update="true|false"
  polymorphism="implicit|explicit"
  where="arbitrary sql where condition"
  persister="PersisterClass"
  batch-size="1"
  optimistic-lock="none|version|dirty|all"
  lazy="true|false"
  entity-name="ClassName as Default"
  check="arbitrary sql check condition"
  rowid="rowid"
  subselect="SQL expression"
  abstract="true|false"
  node="elementName"
/>
```

- 1 name (optional): the fully qualified Java class name of the persistent class or interface. If this attribute is missing, it is assumed that the mapping is for a non-POJO entity.
- 2 table (optional - defaults to the unqualified class name): the name of its database table.
- 3 discriminator-value (optional - defaults to the class name): a value that distinguishes individual subclasses that is used for polymorphic behavior. Acceptable values include null and not null.
- 4 mutable (optional - defaults to true): specifies that instances of the class are (not) mutable. Immutable classes may not be updated or deleted by the application.
- 5 schema (optional): overrides the schema name specified by the root `<hibernate-mapping>` element.

- ⑥ catalog (optional): overrides the catalog name specified by the root <hibernate-mapping>element.
- ⑦ proxy (optional): specifies an interface to use for lazy initializing proxies. You can specify the name of the class itself.
- ⑧ dynamic-update (optional - defaults to false): specifies that UPDATE SQL should be generated at runtime and can contain only those columns whose values have changed.
- ⑨ dynamic-insert (optional - defaults to false): specifies that INSERT SQL should be generated at runtime and contain only the columns whose values are not null.
- ⑩ select-before-update (optional - defaults to false): specifies that Hibernate should *never* perform an SQL UPDATE unless it is certain that an object is actually modified. Only when a transient object has been associated with a new session using update(), will Hibernate perform an extra SQL SELECT to determine if an UPDATE is actually required.
- ⑪ polymorphism (optional - defaults to implicit): determines whether implicit or explicit query polymorphism is used. Use polymorphism="explicit" in the class mapping. This will cause queries to return only instances of the named class and not its subclasses.
- ⑫ where (optional): specifies an arbitrary SQL WHERE condition to be used when retrieving objects of this class.
- ⑬ persister (optional): specifies a custom ClassPersister.
- ⑭ batch-size (optional - defaults to 1): specifies a "batch size" for fetching instances of this class by identifier.
- ⑮ optimistic-lock (optional - defaults to version): determines the optimistic locking strategy.
- (16) lazy (optional): lazy fetching can be disabled by setting lazy="false".
- (17) entity-name (optional - defaults to the class name): Hibernate3 allows a class to be mapped multiple times, potentially to different tables. It also allows entity mappings that are represented by Maps or XML at the Java level. In these cases, you should provide an explicit arbitrary name for the entity.
- (18) check (optional): an SQL expression used to generate a multi-row *check* constraint for automatic schema generation.
- (19) rowid (optional): Hibernate can use ROWIDs on databases. On Oracle, for example, Hibernate can use the rowid extra column for fast updates once this option has been set to rowid. A ROWID is an implementation detail and represents the physical location of a stored tuple.
- (20) subselect (optional): maps an immutable and read-only entity to a database subselect. This is useful if you want to have a view instead of a base table. See below for more information.
- (21) abstract (optional): is used to mark abstract superclasses in <union-subclass> hierarchies.

### **Cascade and Inverse option in One to Many mapping**

Cascade enables operations to cascade to child entities. The valid values are all, none, save-update, delete and all-delete-orphan. The default value is none.

Inverse indicates which end of a bi-directional association relationship should be ignored. It determines whether ask a parent for their children or a child for their parents when persisting a parent who has a collection of children. The valid values are true and false. The default value is false.

## Dynamic Insert versus Dynamic Update

- `dynamic-update` (defaults to `false`): Specifies that `UPDATE` SQL should be generated at runtime and contain only those columns whose values have changed.
- `dynamic-insert` (defaults to `false`): Specifies that `INSERT` SQL should be generated at runtime and contain only those columns whose values are not null.

## Hibernate Proxy

The proxy attribute enables lazy initialization of persistent instances of the class. Hibernate will initially return CGLIB proxies which implement the named interface. The actual persistent object will be loaded when a method of the proxy is invoked.

## Callback Interface

Callback interfaces allow the application to receive a notification when something interesting happens to an object. It happens when an object is loaded, saved, or deleted. Hibernate applications don't need to implement these callbacks, but they're useful for implementing certain kinds of generic functionality.

**HibernateCallback** is the callback interface for Hibernate application. To be used with `HibernateTemplate`'s execution methods, often as anonymous classes within a method implementation. The typical implementation will call `Session.load/find/update` to perform some operations on persistent objects. It can also perform direct JDBC operations via Hibernate's `Session.connection()`, operating on a `JDBC Connection`. The only method that needs to be overridden is **`doInHibernate(session)`** that gets called by `HibernateTemplate`'s `execute` method with an active Hibernate session.

## Session Factory

The application obtains `Session` instances from a `SessionFactory`. There is typically a single `SessionFactory` for the whole application created during application initialization. The `SessionFactory` caches generated SQL statements and other mapping metadata that Hibernate uses at runtime. It also holds cached data that has been read in one unit of work and may be reused in a future unit of work.

Session Factory is a Hibernate's concept of a single data store and is **thread safe** so that many threads can access it concurrently and request for sessions and immutable cache of compiled mappings for a single database. It usually only built once at startup. It should be wrapped in some kind of singleton.

### XML Configuration

```
SessionFactory sessionFactory = new Configuration().configure().buildSessionFactory();
```

### Annotated Configuration

```
SessionFactory sessionFactory = new  
AnnotationConfiguration().configure().addPackage("package.name").addAnnotatedClass(ClassName.class).buildSessionFactory();
```

## Session

The `Session` interface is the primary interface used by Hibernate applications. It is a single-threaded, short-lived object representing a conversation between the application and the persistent store. It allows you to create query objects to retrieve persistent objects.

```
Session session = sessionFactory.openSession();
```

Session interface role:

- Wraps a JDBC connection
- Factory for Transaction
- Holds a mandatory first-level cache of persistent objects, used when navigating the object graph or looking up objects by identifier

Session is a light weight and **non-thread safe** object that represents a single unit of work with the database. Sessions are created by a Session Factory and they are closed when all work is complete. Session is the primary interface of the persistence service. A session obtains a database connection **lazily**. To avoid creating too many sessions, ThreadLocal class can be used.

```
ThreadLocal local = new ThreadLocal().get();  
Session session = sessionFactory.openSession();  
local.set(session);
```

Ending a session usually involves four distinct phases:

1. Flush the session
2. Commit the transaction
3. Handle exceptions and Rollback the transaction
4. Finally, close the session

#### ***Session Interface Methods:***

There are number of methods provided by the **Session** interface but I'm going to list down a few important methods only.

| # | Session Methods and Description                                                                                                        |
|---|----------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>boolean isOpen()</b><br>Check if the session is still open.                                                                         |
| 2 | <b>boolean isConnected()</b><br>Check if the session is currently connected.                                                           |
| 3 | <b>boolean isDirty()</b><br>Does this session contain any changes which must be synchronized with the database?                        |
| 4 | <b>void refresh(Object object)</b><br>Re-read the state of the given instance from the underlying database.                            |
| 5 | <b>void flush()</b><br>Flushing is the process of synchronizing the underlying persistent store with persistable state held in memory. |
| 6 | <b>void clear()</b><br>Completely clear the session.                                                                                   |
| 7 | <b>Connection close()</b><br>End the session by releasing the JDBC connection and cleaning up.                                         |
| 8 | <b>SessionFactory getSessionFactory()</b><br>Get the session factory which created this session.                                       |

|    |                                                                                                                                                                                                                                         |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9  | <b>Serializable getIdentifier(Object object)</b><br>Return the identifier value of the given entity as associated with this session.                                                                                                    |
| 10 | <b>Transaction beginTransaction()</b><br>Begin a unit of work and return the associated Transaction object.                                                                                                                             |
| 11 | <b>Transaction getTransaction()</b><br>Get the Transaction instance associated with this session.                                                                                                                                       |
| 12 | <b>Criteria createCriteria(Class persistentClass)</b><br>Create a new Criteria instance, for the given entity class, or a superclass of an entity class.                                                                                |
| 13 | <b>Criteria createCriteria(String entityName)</b><br>Create a new Criteria instance, for the given entity name.                                                                                                                         |
| 14 | <b>Query createFilter(Object collection, String hqlQueryString)</b><br>Create a new instance of Query for the given collection and filter string.                                                                                       |
| 15 | <b>Query createQuery(String hqlQueryString)</b><br>Create a new instance of Query for the given HQL query string.                                                                                                                       |
| 16 | <b>SQLQuery createSQLQuery(String queryString)</b><br>Create a new instance of SQLQuery for the given SQL query string.                                                                                                                 |
| 17 | <b>void cancelQuery()</b><br>Cancel the execution of the current query.                                                                                                                                                                 |
| 18 | <b>Object load(String entityName, Serializable id, LockOptions lockOptions)</b><br>Return the persistent instance of the given entity class with the given identifier, obtaining the specified lock mode, assuming the instance exists. |
| 19 | <b>Session get(String entityName, Serializable id)</b><br>Return the persistent instance of the given named entity with the given identifier, or null if there is no such persistent instance.                                          |
| 20 | <b>Serializable save(Object object)</b><br>Persist the given transient instance, first assigning a generated identifier.                                                                                                                |
| 21 | <b>void saveOrUpdate(Object object)</b><br>Either save(Object) or update(Object) the given instance.                                                                                                                                    |
| 22 | <b>void update(Object object)</b><br>Update the persistent instance with the identifier of the given detached instance.                                                                                                                 |
| 23 | <b>void update(String entityName, Object object)</b><br>Update the persistent instance with the identifier of the given detached instance.                                                                                              |
| 24 | <b>Object merge(String entityName, Object object)</b><br>Copy the state of the given object onto the persistent object with the same identifier in session.                                                                             |
| 25 | <b>void delete(Object object)</b><br>Remove a persistent instance from the datastore.                                                                                                                                                   |
| 26 | <b>void delete(String entityName, Object object)</b><br>Remove a persistent instance from the datastore.                                                                                                                                |

### Differences between load and get methods

| Load                                                                                                   | Get                                                                            |
|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Only use the load() method if you are sure that the object exists.                                     | If you are not sure that the object exists, then use one of the get() methods. |
| load() method will throw <b>ObjectNotFoundException</b> if the unique id is not found in the database. | get() method will return null if the unique id is not found in the database.   |
| load() just returns a proxy by default and database won't be hit until the proxy is first invoked.     | get() will hit the database immediately.                                       |

### Differences between update and merge methods

| Update                                                                                                                        | Merge                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use update method, if you are sure that the session does not contain an already persistent instance with the same identifier. | Use merge method, if you want to merge your modifications at any time without consideration of the state of the session.                                                                                         |
| It will throw <b>NonUniqueObjectException</b> if a persistent object with the same identifier is already in the session       | It will act like update but if a persistent object with the same identifier is already in the session it will copy the state of the given object onto the persistent object with the same identifier in session. |

### Hibernate Annotations:

#### **@Entity Annotation:**

The EJB 3 standard annotations are contained in the **javax.persistence** package, so we import this package as the first step. Secondly, we used the **@Entity** annotation to the Employee class which marks this class as an entity bean, so it must have a no-argument constructor that is visible with at least protected scope.

#### **@Table Annotation:**

The @Table annotation allows you to specify the details of the table that will be used to persist the entity in the database.

The @Table annotation provides four attributes, allowing you to override the name of the table, its catalogue, and its schema, and enforce unique constraints on columns in the table. For now we are using just table name which is EMPLOYEE.

#### **@Id and @GeneratedValue Annotations:**

Each entity bean will have a primary key, which you annotate on the class with the **@Id** annotation. The primary key can be a single field or a combination of multiple fields depending on your table structure. By default, the @Id annotation will automatically determine the most appropriate primary key generation strategy to be used but you can override this by applying the **@GeneratedValue** annotation which takes two parameters **strategy** and **generator**. We can use the default key generation strategy. Letting Hibernate determine which generator type to use makes your code portable between different databases.

### **@Column Annotation:**

The @Column annotation is used to specify the details of the column to which a field or property will be mapped. You can use column annotation with the following most commonly used attributes:

- **name** attribute permits the name of the column to be explicitly specified.
- **length** attribute permits the size of the column used to map a value particularly for a String value.
- **nullable** attribute permits the column to be marked NOT NULL when the schema is generated.
- **unique** attribute permits the column to be marked as containing only unique values.

### **@OneToOne, @OneToMany, @ManyToOne and @ManyToMany Annotations**

The parameters used by these annotations are fetch (FetchType.LAZY or FetchType.EAGER), cascade (CascadeType.ALL or CascadeType.NONE), inverse (true or false) and orphanRemoval (true or false).

Every relationship that finishes with **@Many** (@OneToMany and @ManyToMany) will be **lazy loaded by default**. Every relationship that finishes with **@One** (@ManyToOne and @OneToOne) will be **eagerly loaded by default**. If you want to set a basic field (e.g. String name) with lazy loading just does:

@Basic (fetch=FetchType.LAZY)

Every basic field (e.g. String, int, double) that we can find inside a class will be eagerly loaded if the developer does not set it as lazy.

### **Entity Class:**

```
@Entity
@Table(name = "EMPLOYEE")
public class Employee {
    @Id @GeneratedValue
    @Column(name = "id")
    private int id;

    @Column(name = "emp_name")
    private String employeeName;

    @Basic(fetch = FetchType.LAZY)
    @Column(name = "salary")
    private int salary;
}
```

### **Application Class:**

```
private static SessionFactory factory;
try{
    factory = new AnnotationConfiguration(). configure().
        addPackage("com.xyz"). //add package if used.
        addAnnotatedClass(Employee.class). buildSessionFactory();
}catch (Throwable ex) {
    System.err.println("Failed to create sessionFactory object." + ex);
    throw new ExceptionInInitializerError(ex);
}

/* Method to CREATE an employee in the database */
```

```

Session session = factory.openSession();
Transaction tx = null;
Integer employeeID = null;
try{
    tx = session.beginTransaction();
    Employee employee = new Employee();
    employee.setFirstName(fname);
    employeeID = (Integer) session.save(employee);
    session.flush();
    tx.commit();
}catch (HibernateException e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}

```

```

/* Method to READ all the employees */
Session session = factory.openSession();
Transaction tx = null;
try{
    tx = session.beginTransaction();
    session.setFlushMode(FlushMode.COMMIT);
    List employees = session.createQuery("FROM Employee").list();
    for (Iterator iterator = employees.iterator(); iterator.hasNext();){
        Employee employee = (Employee) iterator.next();
    }
    tx.commit(); //flush occurs here
}catch (HibernateException e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}

```

```

/* Method to UPDATE salary for an employee */
Session session = factory.openSession();
Transaction tx = null;
try{
    tx = session.beginTransaction();
    Employee employee = (Employee)session.get(Employee.class, EmployeeID);
    employee.setSalary( salary );
    session.update(employee);
    session.flush();
    tx.commit();
}catch (HibernateException e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}
/* Method to DELETE an employee from the records */
Session session = factory.openSession();
Transaction tx = null;

```

```

try{
    tx = session.beginTransaction();
    session.setFlushMode(FlushMode.COMMIT);
    Employee employee = (Employee)session.get(Employee.class, EmployeeID);
    session.delete(employee);
    tx.commit(); //flush occurs here
}catch (HibernateException e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}

```

## Query Interface

### HQL (Hibernate Query Language)

*Hibernate Query Language (HQL) is an object-oriented query language, similar to SQL, but instead of operating on tables and columns, HQL works with persistent objects and their properties. HQL queries are translated by Hibernate into conventional SQL queries which in turn perform actions on database tables.*

Although you can use SQL statements directly with Hibernate using Native SQL but I would recommend using HQL whenever possible to avoid database portability hassles, and to take advantage of Hibernate's SQL generation and caching strategies.

Keywords like SELECT, FROM and WHERE etc. are not case sensitive but properties like table and column names are case sensitive in HQL.

### FROM Clause

You will use **FROM** clause if you want to load a complete persistent objects into memory. Following is the simple syntax of using FROM clause:

```

String hql = "FROM Employee";
Query query = session.createQuery(hql);
List results = query.list();

```

If you need to fully qualify a class name in HQL, just specifies the package and class name as follows:

```

String hql = "FROM com.mesonsoft.criteria.Employee";
Query query = session.createQuery(hql);
List results = query.list();

```

## AS Clause

In HQL, the **AS** clause can be used to assign aliases to the classes, especially when you have long queries. For instance, our previous simple example would be the following:

```
String hql = "FROM Employee AS E";  
Query query = session.createQuery(hql);  
List results = query.list();
```

The **AS** keyword is optional and you can also specify the alias directly after the class name, as follows:

```
String hql = "FROM Employee E";  
Query query = session.createQuery(hql);  
List results = query.list();
```

## SELECT Clause

The **SELECT** clause provides more control over the result set than the FROM clause. If you want to obtain few properties of objects instead of the complete object, use the SELECT clause. Following is the simple syntax of using SELECT clause to get just first\_name field of the Employee object:

```
String hql = "SELECT E.firstName FROM Employee E";  
Query query = session.createQuery(hql);  
List results = query.list();
```

It is notable here that **Employee.firstName** is a property of Employee object rather than a field of the EMPLOYEE table.

## WHERE Clause

If you want to narrow the specific objects that are returned from storage, you use the WHERE clause. Following is the simple syntax of using WHERE clause:

```
String hql = "FROM Employee E WHERE E.id = 10";  
Query query = session.createQuery(hql);  
List results = query.list();
```

## ORDER BY Clause

To sort your HQL query's results, you will need to use the **ORDER BY** clause. You can order the results by any property on the objects in the result set either ascending (ASC) or descending (DESC). Following is the simple syntax of using ORDER BY clause:

```
String hql = "FROM Employee E WHERE E.id > 10 ORDER BY E.salary DESC";  
Query query = session.createQuery(hql);  
List results = query.list();
```

If you wanted to sort by more than one property, you would just add the additional properties to the end of the order by clause, separated by commas as follows:

```
String hql = "FROM Employee E WHERE E.id > 10 ORDER BY E.firstName DESC, E.salary DESC ";
Query query = session.createQuery(hql);
List results = query.list();
```

### GROUP BY Clause

This clause lets Hibernate pull information from the database and group it based on a value of an attribute and, typically, uses the result to include an aggregate value. Following is the simple syntax of using GROUP BY clause:

```
String hql = "SELECT SUM(E.salary), E.firtName FROM Employee E GROUP BY E.firstName";
Query query = session.createQuery(hql);
List results = query.list();
```

### Using Named Parameters

Hibernate supports named parameters in its HQL queries. This makes writing HQL queries that accept input from the user easy and you do not have to **defend against SQL injection attacks**. Following is the simple syntax of using named parameters:

```
String hql = "FROM Employee E WHERE E.id = :employee_id";
Query query = session.createQuery(hql);
query.setParameter("employee_id",10);
List results = query.list();
```

### Bulk Updates

Bulk updates are new to HQL with Hibernate 3, and deletes work differently in Hibernate 3 than they did in Hibernate 2. The Query interface now contains a new method called executeUpdate() for executing HQL UPDATE, INSERT or DELETE statements.

### UPDATE Clause

The **UPDATE** clause can be used to update one or more properties of one or more objects. Following is the simple syntax of using UPDATE clause:

```
String hql = "UPDATE Employee set salary = :salary WHERE id = :employee_id";
Query query = session.createQuery(hql);
query.setParameter("salary", 1000);
query.setParameter("employee_id", 10);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

## DELETE Clause

The **DELETE** clause can be used to delete one or more objects. Following is the simple syntax of using DELETE clause:

```
String hql = "DELETE FROM Employee WHERE id = :employee_id";
Query query = session.createQuery(hql);
query.setParameter("employee_id", 10);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

## INSERT Clause

HQL supports **INSERT INTO** clause only where records can be inserted from one object to another object. Following is the simple syntax of using INSERT INTO clause:

```
String hql = "INSERT INTO Employee(firstName, lastName, salary)" +
            "SELECT firstName, lastName, salary FROM old_employee";
Query query = session.createQuery(hql);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

## Aggregate Methods

HQL supports a range of aggregate methods, similar to SQL. They work the same way in HQL as in SQL and following is the list of the available functions:

| S.N. | Functions                 | Description                                          |
|------|---------------------------|------------------------------------------------------|
| 1    | avg(property name)        | The average of a property's value                    |
| 2    | count(property name or *) | The number of times a property occurs in the results |
| 3    | max(property name)        | The maximum value of the property values             |
| 4    | min(property name)        | The minimum value of the property values             |
| 5    | sum(property name)        | The sum total of the property values                 |

## DISTINCT

The **distinct** keyword only counts the unique values in the row set. The following query will return only unique count:

```
String hql = "SELECT count(DISTINCT E.firstName) FROM Employee E";
Query query = session.createQuery(hql);
List results = query.list();
```

## Pagination using Query

There are two methods of the Query interface for pagination.

| S.N. | Method & Description                                                                                                                                  |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>Query setFirstResult(int startPosition)</b><br>This method takes an integer that represents the first row in your result set, starting with row 0. |
| 2    | <b>Query setMaxResults(int maxCount)</b><br>This method tells Hibernate to retrieve a fixed number <b>maxCount</b> of objects.                        |

Using above two methods together, we can construct a paging component in our web or Swing application. Following is the example which you can extend to fetch 10 rows at a time:

```
String hql = "FROM Employee";
Query query = session.createQuery(hql);
query.setFirstResult(1);
query.setMaxResults(10);
List results = query.list();
```

## Ways to express joins in HQL

HQL provides four ways of expressing equi (inner and outer) joins:-

- An **implicit** association join
- An ordinary JOIN in the FROM clause (RIGHT JOIN)
- A FETCH JOIN in the FROM clause. (LEFT JOIN FETCH)
- A **theta-style** join in the WHERE clause. (to find all children of an entity)

Select c from Entity e, Child c **where e = c.parent**

Such joins are called theta style joins - a cartesian product + a where condition to restrict the result.

## Criteria Interface

'Criteria' is a simplified API for retrieving entities by composing Criterion objects and build up a criteria query object programmatically where you can apply **filtration rules and logical conditions**. This is a very convenient approach for functionality like "search" screens where there are a variable number of conditions to be placed upon the result set.

The Hibernate **Session** interface provides **createCriteria()** method which can be used to create a **Criteria** object that returns instances of the persistence object's class when your application executes a criteria query.

Following is the simplest example of a criteria query is one which will simply return every object that corresponds to the Employee class.

```
Criteria cr = session.createCriteria(Employee.class);
List results = cr.list();
```

### Restrictions with Criteria:

You can use **add()** method available for **Criteria** object to add restriction for a criteria query. Following is the example to add a restriction to return the records with salary is equal to 2000:

```
Criteria cr = session.createCriteria(Employee.class);
cr.add(Restrictions.eq("salary", 2000));
List results = cr.list();
```

Following are the few more examples covering different scenarios and can be used as per requirement:

```
Criteria cr = session.createCriteria(Employee.class);

// To get records having salary equals to 2000
cr.add(Restrictions.eq("salary", 2000));

// To get records having salary more than 2000
cr.add(Restrictions.gt("salary", 2000));

// To get records having salary less than 2000
cr.add(Restrictions.lt("salary", 2000));

// To get records having firstName starting with zara
cr.add(Restrictions.like("firstName", "zara%"));

// Case sensitive form of the above restriction.
cr.add(Restrictions.ilike("firstName", "zara%"));

// To get records having salary in between 1000 and 2000
cr.add(Restrictions.between("salary", 1000, 2000));

// To check if the given property is null
cr.add(Restrictions.isNull("salary"));

// To check if the given property is not null
cr.add(Restrictions.isNotNull("salary"));

// To check if the given property is empty
cr.add(Restrictions.isEmpty("salary"));

// To check if the given property is not empty
cr.add(Restrictions.isNotEmpty("salary"));
```

You can create AND or OR conditions using LogicalExpression restrictions as follows:

```
Criteria cr = session.createCriteria(Employee.class);

Criterion salary = Restrictions.gt("salary", 2000);
Criterion name = Restrictions.ilike("firstName", "zara%");

// To get records matching with OR conditions
LogicalExpression orExp = Restrictions.or(salary, name);
cr.add( orExp );

// To get records matching with AND conditions
LogicalExpression andExp = Restrictions.and(salary, name);
cr.add( andExp );

List results = cr.list();
```

Though, all the above conditions can be used directly with HQL as explained before.

#### Pagination using Criteria:

There are two methods of the Criteria interface for pagination.

| S.N. | Method & Description                                                                                                                                            |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>public Criteria setFirstResult(int startPosition)</b><br>This method takes an integer that represents the first row in your result set, starting with row 0. |
| 2    | <b>public Criteria setMaxResults(int maxCount)</b><br>This method tells Hibernate to retrieve a fixed number <b>maxResults</b> of objects.                      |

Using above two methods together, we can construct a paging component in our web or Swing application. Following is the example which you can extend to fetch 10 rows at a time:

```
Criteria cr = session.createCriteria(Employee.class);
cr.setFirstResult(1);
cr.setMaxResults(10);
List results = cr.list();
```

#### Sorting the Results:

The Criteria API provides the **org.hibernate.criterion.Order** class to sort your result set in either ascending or descending order, according to one of your object's properties. This example demonstrates how you would use the Order class to sort the result set:

```
Criteria cr = session.createCriteria(Employee.class);
// To get records having salary more than 2000
cr.add(Restrictions.gt("salary", 2000));

// To sort records in descening order
```

```
crit.addOrder(Order.desc("salary"));

// To sort records in ascending order
crit.addOrder(Order.asc("salary"));

List results = cr.list();
```

### Projections & Aggregations:

The Criteria API provides the **org.hibernate.criterion.Projections** class which can be used to get average, maximum or minimum of the property values. The Projections class is similar to the Restrictions class in that it provides several static factory methods for obtaining **Projection** instances.

Following are the few examples covering different scenarios and can be used as per requirement:

```
Criteria cr = session.createCriteria(Employee.class);

// To get total row count.
cr.setProjection(Projections.rowCount());

// To get average of a property.
cr.setProjection(Projections.avg("salary"));

// To get distinct count of a property.
cr.setProjection(Projections.countDistinct("firstName"));

// To get maximum of a property.
cr.setProjection(Projections.max("salary"));

// To get minimum of a property.
cr.setProjection(Projections.min("salary"));

// To get sum of a property.
cr.setProjection(Projections.sum("salary"));
```

### Hibernate Native SQL

You can use native SQL to express database queries if you want to utilize database-specific features such as query hints or the CONNECT keyword in Oracle. Hibernate 3.x allows you to specify handwritten SQL, including stored procedures, for all create, update, delete, and load operations.

Your application will create a native SQL query from the session with the **createSQLQuery()** method on the Session interface.:

```
public SQLQuery createSQLQuery(String sqlStatement) throws HibernateException
```

After you pass a string containing the SQL query to the `createSQLQuery()` method, you can associate the SQL result with either an existing Hibernate entity, a join, or a scalar result using `addEntity()`, `addJoin()`, and `addScalar()` methods respectively.

#### Scalar queries:

The most basic SQL query is to get a list of scalars (values) from one or more tables. Following is the syntax for using native SQL for scalar values:

```
String sql = "SELECT first_name, salary FROM EMPLOYEE";
SQLQuery query = session.createSQLQuery(sql);
query.setResultTransformer(Criteria.ALIAS_TO_ENTITY_MAP);
List results = query.list();
```

#### Entity queries:

The above queries were all about returning scalar values, basically returning the "raw" values from the resultset. The following is the syntax to get entity objects as a whole from a native sql query via `addEntity()`.

```
String sql = "SELECT * FROM EMPLOYEE";
SQLQuery query = session.createSQLQuery(sql);
query.addEntity(Employee.class);
List results = query.list();
```

#### Named SQL queries:

The following is the syntax to get entity objects from a native sql query via `addEntity()` and using named SQL query.

```
String sql = "SELECT * FROM EMPLOYEE WHERE id = :employee_id";
SQLQuery query = session.createSQLQuery(sql);
query.addEntity(Employee.class);
query.setParameter("employee_id", 10);
List results = query.list();
```

Named SQL queries are defined in the mapping XML document and called wherever required.

#### Example:

```
<sql-query name = "empdetails">
  <return alias="emp" class="com.test.Employee"/>
  SELECT emp.EMP_ID AS {emp.empid},
         emp.EMP_ADDRESS AS {emp.address},
         emp.EMP_NAME AS {emp.name}
  FROM Employee EMP WHERE emp.NAME LIKE :name
</sql-query>
```

Invoke Named Query :

```
List people = session.getNamedQuery("empdetails")
                        .setString("TomBrady", name)
                        .setMaxResults(50)
                        .list();
```

### Invoke Stored Procedure

```
<sql-query name="selectAllEmployees_SP" callable="true">
<return alias="emp" class="employee">
  <return-property name="empid" column="EMP_ID"/>
  <return-property name="name" column="EMP_NAME"/>
  <return-property name="address" column="EMP_ADDRESS"/>
  { ? = call selectAllEmployees() }
</return>
</sql-query>
```

### Hibernate Template

*org.springframework.orm.hibernate.HibernateTemplate* is a helper class which provides different methods for querying and retrieving data from the database. It also converts the checked *HibernateExceptions* into unchecked *DataAccessExceptions* (Runtime Exceptions).

The benefits of *HibernateTemplate* are:

- *HibernateTemplate*, a Spring Template class simplifies interactions with Hibernate session.
- Common functions are simplified to a single method call.
- Sessions are automatically closed.
- Exceptions are automatically caught and converted into runtime exceptions.

#### Tips:

- Using Hibernate SQL Dialects, we can switch databases. Hibernate will generate appropriate hql queries based on the dialect defined.
- In order to print the Hibernate generated SQL statements on Console, the configuration file has to be updated with the following

```
<property name="show_sql">true</property>
```

## Collection Types in Hibernate

Collection Types are:

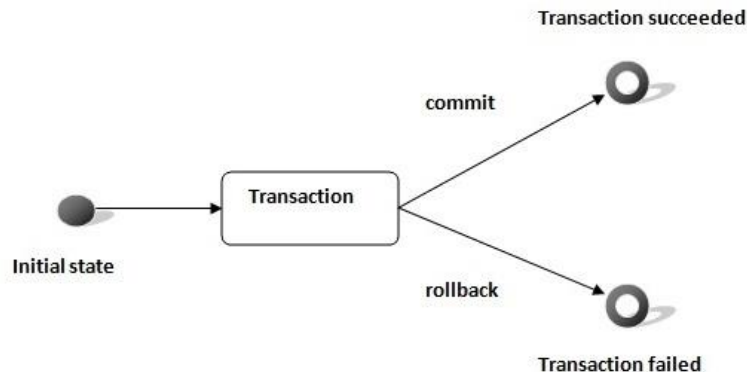
- Bag
- Set
- List
- Array
- Map

## Sorted Collection versus Ordered Collection

| Sorted Collection                                                                                                                                                                                                                                     | Ordered Collection                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| A sorted collection is sorting a collection by utilizing the sorting features provided by the Java collections framework. The sorting occurs in the memory of JVM that runs Hibernate, after the data being read from database using java comparator. | Order collection is sorting a collection by specifying the ORDER BY clause for sorting this collection when retrieval. |
| If your collection is not large, it will be more efficient way to sort it.                                                                                                                                                                            | If your collection is very large, it will be more efficient way to sort it.                                            |

## Hibernate Transaction Management

A **transaction** simply represents a unit of work. In such case, if one step fails, the whole transaction fails (which is termed as atomicity). A transaction can be described by ACID properties (Atomicity, Consistency, Isolation and Durability).



## Transaction Interface

In hibernate framework, we have **Transaction** interface that defines the unit of work. It maintains abstraction from the transaction implementation (JTA, JDBC, so on).

A transaction is associated with Session and instantiated by calling **session.beginTransaction()**.

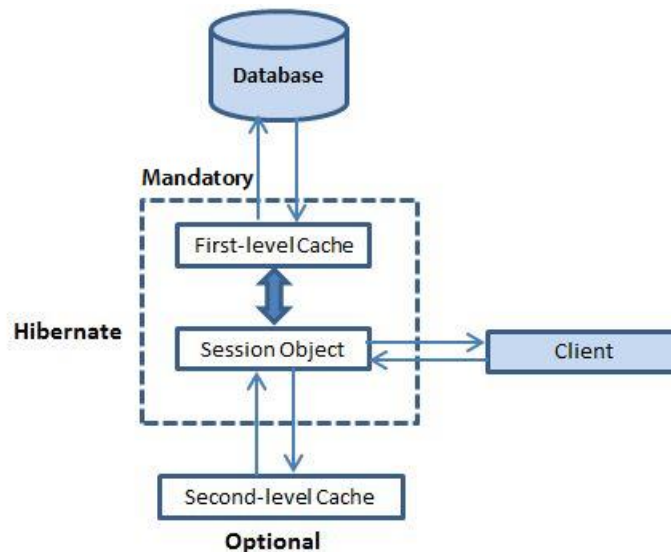
The methods of Transaction interface are as follows:

1. **void begin()** starts a new transaction.
2. **void commit()** ends the unit of work unless we are in **FlushMode.NEVER**.
3. **void rollback()** forces this transaction to rollback.
4. **void setTimeout(int seconds)** it sets a transaction timeout for any transaction started by a subsequent call to begin on this instance.
5. **void registerSynchronization(Synchronization s)** registers a user synchronization callback for this transaction.
6. **boolean isAlive()** checks if the transaction is still alive.
7. **boolean wasCommitted()** checks if the transaction is committed successfully.
8. **boolean wasRolledBack()** checks if the transaction is rolledback successfully.

## Caching

Caching is all about application performance optimization and it sits between your application and the database to avoid the number of database hits as many as possible to give a better performance for performance critical applications.

Caching is important to Hibernate as well which utilizes multilevel caching schemes as explained below:



### First-level cache:

The first-level cache is the Session cache and is a mandatory cache through which all requests must pass. The Session object keeps an object under its own power before committing it to the database.

If you issue multiple updates to an object, Hibernate tries to delay doing the update as long as possible to reduce the number of update SQL statements issued. If you close the session, all the objects being cached are lost and either persisted or updated in the database.

### Second-level cache:

Second level cache is an optional cache and first-level cache will always be consulted before any attempt is made to locate an object in the second-level cache. The second-level cache can be configured on a **per-class** and **per-collection basis** and mainly responsible for **caching objects across sessions**.

Any third-party cache can be used with Hibernate. An **org.hibernate.cache.CacheProvider** interface is provided, which must be implemented to provide Hibernate with a handle to the cache implementation.

### Query-level cache:

Hibernate also implements a cache for query result sets that integrates closely with the second-level cache.

This is an optional feature and requires two additional physical cache regions that hold the **cached query results and the timestamps** when a table was last updated. This is only useful for queries that are **run frequently with the same parameters**.

To use the query cache, you must first activate it using the **hibernate.cache.use\_query\_cache = "true"** property in the configuration file. By setting this property to true, you make Hibernate create the necessary caches in memory to hold the query and identifier sets.

Next, to use the query cache, you use the `setCacheable(Boolean)` method of the Query class. For example:

```
Session session = SessionFactory.openSession();
Query query = session.createQuery("FROM EMPLOYEE");
query.setCacheable(true);
List users = query.list();
SessionFactory.closeSession();
```

Hibernate also supports very fine-grained cache support through the concept of a cache region. A cache region is part of the cache that's given a name.

```
Session session = SessionFactory.openSession();
Query query = session.createQuery("FROM EMPLOYEE");
query.setCacheable(true);
query.setCacheRegion("employee");
List users = query.list();
SessionFactory.closeSession();
```

This code uses the method to tell Hibernate to store and look for the query in the employee area of the cache.

### The Second Level Cache:

Hibernate uses first-level cache by default and you have nothing to do to use first-level cache. Let's go straight to the optional second-level cache. Not all classes benefit from caching, so it's important to be able to disable the second-level cache

The Hibernate second-level cache is set up in two steps. First, you have to decide which concurrency strategy to use. After that, you configure cache expiration and physical cache attributes using the cache provider.

### Concurrency strategies:

A concurrency strategy is a mediator which responsible for storing items of data in the cache and retrieving them from the cache. If you are going to enable a second-level cache, you will have to decide, for each persistent class and collection, which cache concurrency strategy to use.

- **Transactional:** Use this strategy for read-mostly data where it is critical to prevent stale data in concurrent transactions, in the rare case of an update.
- **Read-write:** Again use this strategy for read-mostly data where it is critical to prevent stale data in concurrent transactions, in the rare case of an update.
- **Non-strict-read-write:** This strategy makes no guarantee of consistency between the cache and the database. Use this strategy if data hardly ever changes and a small likelihood of stale data is not of critical concern.
- **Read-only:** A concurrency strategy suitable for data which never changes. Use it for reference data only.

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
  <class name="Employee" table="EMPLOYEE">
    <meta attribute="class-description">
      This class contains the employee detail.
    </meta>
    <cache usage="read-write"/>
    <id name="id" type="int" column="id">
      <generator class="native"/>
    </id>
    <property name="firstName" column="first_name" type="string"/>
    <property name="lastName" column="last_name" type="string"/>
    <property name="salary" column="salary" type="int"/>
  </class>
</hibernate-mapping>
```

The usage="read-write" attribute tells Hibernate to use a read-write concurrency strategy for the defined cache.

### Cache provider:

Your next step after considering the concurrency strategies you will use for your cache candidate classes is to pick a cache provider. **Hibernate forces you to choose a single cache provider for the whole application.**

Every cache provider is not compatible with every concurrency strategy. The following compatibility matrix will help you choose an appropriate combination.

| Strategy/Provider | Read-only | Non-strict-read-write | Read-write | Transactional |
|-------------------|-----------|-----------------------|------------|---------------|
| EHCache           | X         | X                     | X          |               |
| OSCache           | X         | X                     | X          |               |
| SwarmCache        | X         | X                     |            |               |
| JBoss Cache       | X         |                       |            | X             |

## Batch Processing

Consider a situation when you need to upload a large number of records into your database using Hibernate. Following is the code snippet to achieve this using Hibernate:

```
Session session = SessionFactory.openSession();
Transaction tx = session.beginTransaction();
for ( int i=0; i<100000; i++ ) {
    Employee employee = new Employee(.....);
    session.save(employee);
}
tx.commit();
session.close();
```

Because by default, Hibernate will cache all the persisted objects in the session-level cache and ultimately your application would fall over with an **OutOfMemoryException** somewhere around the 50,000th row. You can resolve this problem if you are using **batch processing** with Hibernate.

To use the batch processing feature, first set **hibernate.jdbc.batch\_size** as batch size to a number either at 20 or 50 depending on object size. This will tell the hibernate container that every X rows to be inserted as batch. To implement this in your code we would need to do little modification as follows:

```
Session session = SessionFactory.openSession();
Transaction tx = session.beginTransaction();
for ( int i=0; i<100000; i++ ) {
    Employee employee = new Employee(.....);
    session.save(employee);
    if( i % 50 == 0 ) { // Same as the JDBC batch size
        // Flush a batch of inserts
        session.flush();
        // Release memory
        session.clear();
    }
}
tx.commit();
session.close();
```

Above code will work fine for the INSERT operation, but if you are willing to make UPDATE operation then you can achieve using the following code:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

ScrollableResults employeeCursor = session.createQuery("FROM EMPLOYEE")
    .scroll();
int count = 0;

while ( employeeCursor.next() ) {
    Employee employee = (Employee) employeeCursor.get(0);
    employee.updateEmployee();
    session.update(employee);
    if ( ++count % 50 == 0 ) {
        session.flush();
        session.clear();
    }
}
tx.commit();
session.close();
```

### Batch Processing Example:

Let us modify configuration file as to add **hibernate.jdbc.batch\_size** property:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration SYSTEM
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQLDialect
    </property>
    <property name="hibernate.connection.driver_class">
      com.mysql.jdbc.Driver
    </property>

    <!-- Assume students is the database name -->
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost/test
    </property>
    <property name="hibernate.connection.username">
      root
    </property>
    <property name="hibernate.connection.password">
      root123
    </property>
    <property name="hibernate.jdbc.batch_size">
      50
    </property>

    <!-- List of XML mapping files -->
    <mapping resource="Employee.hbm.xml"/>

  </session-factory>
</hibernate-configuration>
```

## Hibernate Interceptor

As you have learnt that in Hibernate, an object will be created and persisted. Once the object has been changed, it must be saved back to the database. This process continues until the next time the object is needed, and it will be loaded from the persistent store.

Thus an object passes through different stages in its life cycle and **Interceptor Interface** provides methods which can be called at different stages to perform some required tasks. These methods are callbacks from the session to the application, allowing the application to inspect and/or manipulate properties of a persistent object before it is saved, updated, deleted or loaded. Following is the list of all the methods available within the Interceptor interface:

| S.N. | Method and Description                                                                                                                                      |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>instantiate()</b><br>This method is called when a persisted class is instantiated.                                                                       |
| 2    | <b>onLoad()</b><br>This method is called before an object is initialized.                                                                                   |
| 3    | <b>onSave()</b><br>This method is called before an object is saved.                                                                                         |
| 4    | <b>onDelete()</b><br>This method is called before an object is deleted.                                                                                     |
| 5    | <b>findDirty()</b><br>This method is be called when the <b>flush()</b> method is called on a Session object.                                                |
| 6    | <b>isUnsaved()</b><br>This method is called when an object is passed to the <b>saveOrUpdate()</b> method.                                                   |
| 7    | <b>preFlush()</b><br>This method is called before a flush.                                                                                                  |
| 8    | <b>onFlushDirty()</b><br>This method is called when Hibernate detects that an object is dirty (ie. have been changed) during a flush i.e. update operation. |
| 9    | <b>postFlush()</b><br>This method is called after a flush has occurred and an object has been updated in memory.                                            |

Hibernate Interceptor gives us total control over how an object will look to both the application and the database.

## How to use Interceptors?

To build an interceptor you can either implement **Interceptor** interface directly or extend **EmptyInterceptor** class. Following will be the simple steps to use Hibernate Interceptor functionality.

```
/* Method to CREATE an employee in the database */
Session session = factory.openSession( new MyInterceptor() );
Transaction tx = null;
Integer employeeID = null;
try{
    tx = session.beginTransaction();
    Employee employee = new Employee(fname, lname, salary);
    employeeID = (Integer) session.save(employee);
    tx.commit();
}catch (HibernateException e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}
return employeeID;
```

## Inheritance models in Hibernate

There are three types of inheritance models in Hibernate:

1. Table per class hierarchy
2. Table per subclass
3. Table per concrete class

## Fetching Strategy

A fetching strategy is the strategy that Hibernate will use for retrieving associated objects if the application needs to navigate the association. Fetch strategies may be declared in the O/R mapping metadata, or over-ridden by a particular HQL or Query By Criteria (QBC).

### Hibernate Fetching Strategies

1. fetch="join", **@Fetch**(FetchMode.JOIN) or **@Basic**(fetch=FetchType.EAGER)

Disable the **lazy loading**, always load all the collections and entities.

2. fetch="select", (default) **@Fetch**(FetchMode.SELECT) or **@Basic**(fetch=FetchType.LAZY)

Lazy load all the collections and entities when necessary

3. fetch="subselect", **@Fetch**(FetchMode.SUBSELECT)

Group its collection into a sub select statement.

4. batch-size="10", **@BatchSize**(size=10)

Fetch up to 'N' collections or entities, Not Record.

## Automatic Dirty Checking

Automatic dirty checking is a feature that saves us the effort of explicitly asking Hibernate to update the database when we modify the state of an object inside a transaction.

## Transactional Write-Behind

Hibernate uses a sophisticated algorithm to determine an efficient ordering that avoids database foreign key constraint violations but is still sufficiently predictable to the user. This feature is called transactional write-behind.

## Hibernate Instance States

Three types of instance states are:

**Transient (New)** - The instance is not associated with any persistence context

**Persistent** - The instance is associated with a persistence context

**Detached** - The instance was associated with a persistence context which has been closed and currently not associated

## Detached Objects

It can be passed across layers all the way up to the presentation layer without having to use any DTOs. It can be later on re-attached to another session.

## Distinguish between Transient (New) Objects and Detached Objects

- Use version property if exists
- Use the identifier value. No value for new objects. Natural and assigned surrogate keys don't work.
- Own strategy with `Interceptor.isUnsaved()` that will be invoked when `saveOrUpdate` is called.

## Lazy Initialization Exception

This error means that you're trying to access a lazily-loaded property or collection, but the hibernate session is closed or not available. Lazy loading in Hibernate means that the object will not be populated (via a database query) until the property or collection is accessed in code. Hibernate accomplishes this by creating a dynamic proxy object that will hit the database only when you first use the object. In order for this to work, your object must be attached to an open Hibernate session throughout its lifecycle.

One way to fix this problem is to force Hibernate to perform an Eager fetching by using 'fetch = FetchType.EAGER' on the collection of the entity class. Once again, if you are building real world applications this kind of quickfix will not work.

For example, if your entity has one collection of objects, and each one of these objects has more than one collection of objects then writing 'fetch = FetchType.EAGER' next to each collection will probably carry you to have the `PersistenceException: HibernateException: cannot simultaneously fetch multiple bags`. Now, instead of trying to fix our brand new bug, let's try to fix our configuration to avoid having the first `LazyInitializationException` in a clean way.

### How to avoid the LazyInitializationException:

All you need is to maintain the session of your sessionFactory open while the processing the request. This is possible by adding an **OpenSessionInViewFilter** filter in your web.xml that will bind the session to the thread of the entire processing of the request. By doing this, **the filter will maintain the session opened** (during the request handling) **and allows the Lazy fetching**.

```
<filter>
  <filter-name>OpenSessionInViewFilter</filter-name>
  <filter-class>org.springframework.orm.hibernate3.support.OpenSessionInViewFilter</filter-class>
</filter>
<filter-mapping>
  <filter-name>OpenSessionInViewFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

### Pros and Cons of Lazy Loading

Lazy Initialization can help prevent unnecessary SELECT queries being executed when retrieving an object graph. But, with Lazy Loading, we have Select N+1 problem with one-to-many (parent-child) relationships while retrieving an object graph using HQL or Query by Criteria (QBC).

**Solution:** Eager loading by means of proper database table joins. But, the problem with Eager Loading is decreased performance.

### Select N+1 Problem

Select N + 1 is a **data access anti-pattern** where the database is accessed in a suboptimal way. Take a look at this code sample, say you want to show the user all comments from all posts so that they can delete all of the nasty comments. The naive implementation would be something like:

```
// SELECT * FROM Posts
foreach (Post post in session.createQuery("from Post").List()) {
    //lazy loading of comments list causes:
    // SELECT * FROM Comments where PostId = @p0
    foreach (Comment comment in post.Comments) {
        //print comment...
    }
}
```

In this example, we can see that we are loading a list of posts (the first select) and then traversing the object graph. However, we access the collection in a lazy fashion, causing Hibernate to go to the database and bring the results back one row at a time. This is incredibly inefficient.

**The solution for this problem is simple:** Force an eager load of the collection up front.

#### Using HQL:

```
var posts = session.createQuery("FROM Post p LEFT JOIN FETCH p.Comments").List();
```

#### Using the criteria API:

```
session.createCriteria(typeof(Post)).setFetchMode("Comments", FetchType.EAGER).List();
```

In each case, we will get a join and only a **single query** to the database.

## Hibernate – Pointers

### 1. Introduction to problems

- Mapping member variables to columns
- Mapping relationships
- Handling Datatypes
- Managing changes to object state

### 2. JDBC vs Hibernate

- JDBC Database Configuration – Hibernate Configuration file (hibernate.cfg.xml)
- The Model object – Annotations
- Service method to create model objects – Hibernate API
- Database Design – Not Needed
- DAO Method – Not Needed

### 3. Model class with Annotations

- @Entity - Must
- @Id - Must

### 4. Saving model objects

Hibernate Flow (Configuration → Session Factory → Session → Begin Transaction → Operations → Commit Transaction → Error Handling → Rollback Transaction → Finally, Close Session)

### 5. HBM2DDL Configuration and Name Annotations

- @Entity (name) - optional
- @Column (name) – optional
- Create and Update options in HBM2DDL configuration

### 6. Other Model Annotations

- @Table – Customize Table Attributes
- @Column – Customize Column Attributes
- @Basic – Customize Fetch Type
- @Transient – Exclude Field
- @Temporal – Date
- @LOB – Large Objects (Determine Blob or Clob based on the datatype)

### 7. Object Retrieval

- Session.get()
- Session.get() vs Session.load()

## **8. Primary Keys**

- Natural vs Surrogate
- Surrogate - @GeneratedValue(strategy = GenerationType.AUTO, IDENTITY, SEQUENCE and TABLE)

## **9. Value Types or Value Objects and Embedded Objects**

- @Embeddable
- @Embedded

## **10. Attribute Overrides and Embedded Object Keys**

- @AttributeOverrides and @AttributeOverride – Overrides the default @Column attributes
- @EmbeddedId – Composite Primary Key

## **11. Saving Collections**

- @ElementCollection

## **12. Configuring Collections and Adding Keys**

- @JoinTable
- @JoinColumn
- @CollectionId – Primary Key Column – Hibernate Feature (Not in JPA specification)
- @GenericGenerator – Key Generation (hilo)
- @Type

## **13. Proxy Objects and Eager/Lazy Fetch Types**

- @Basic(fetchType = FetchType.EAGER or LAZY)
- @FetchMode(fetchType = FetchType.EAGER or LAZY)
- First Level – Eager and Sub Levels – Lazy
- Lazy Initialization Exception – Fetching the association after the session is closed
- OpenSessionInViewFilter – Prevents Lazy Initialization Exception

## **14. One to One Mapping**

- @Entity @OneToOne – Same Table
- @JoinColumn – Customize the join column attributes

## **15. One to Many Mapping or Many to One Mapping**

- @Entity @OneToMany or @ManyToOne – Separate Table
- @JoinTable – Customize the join table attributes
- @OneToMany(mappedBy = "tableName") – No Separate Join Table
- @JoinColumn – Customize the join column attributes

## 16. Many to Many Mapping

- @Entity @ManyToMany – Two Separate Tables
- @JoinTable – Customize the join table attributes
- @ManyToMany (mappedBy = "TableName") – One Separate Join Table – Prevent two tables
- @JoinColumn – Customize the join column attributes
- @InverseJoinColumn – Customize the inverse join column attributes

## 17. Cascade Types & Entity Relationships

- @Entity(CascadeType.ALL, PERSIST (Multiple Saves), DETACH, MERGE, REFRESH, REMOVE and INVERSE)
- Session.persist()
- @NotFound (ActionType.IGNORE) – Ignores when the associated object is missing

## 18. Hibernate Collections

- Bag (ArrayList)
- Bag with Id (ArrayList)
- List – Ordered and Duplicates(ArrayList)
- Set – Unordered and No Duplicates
- Map – Key/Value Pairs

## 19. Inheritance in Hibernate

- Entity Class extends another Entity Class – DTYPE (Column) – Sub Class Names (Values)
- Strategies – Single Table (Default), Table per Class and Joined

## 20. Single Table Inheritance

- Super Class @Entity @Inheritance (StrategyType.SINGLE\_TABLE)
- Disadvantage: Null Columns
- @Discriminator Column (name, type) – Customize the discriminator column attributes (DTYPE)
- @DiscriminatorValue - Customize the discriminator value (Sub Class Name)

## 21. Table per Class Inheritance

- Separate tables per class – Avoid blank columns but introduce Redundancy with multiple duplicate columns
- @Entity @Inheritance (StrategyType.TABLE\_PER\_CLASS)

## 22. Inheritance with Joined

- @Entity @Inheritance (StrategyType.JOINED) – Prevent Redundancy with duplicate columns

### 23. CRUD Operations

- save or persist – Track down the object state and issue updates wisely (Automatic Dirty Checking)
- get or load – Automatic Dirty Checking
- update or merge
- delete

### 24. Hibernate Instance States

- Transient (New) – not associated with any persistent context
- Persistent (In Session) – associated with a persistent context
- Detached (Removed) – was associated with a persistent context, closed or currently not associated

### 25. Understanding Instance States

- Transient → Persistent → Detached
- New → Transient → Save → Persistent → Close → Detached
- Get → Persistent → Close → Detached
- Transient ← Delete ← Persistent → Close → Detached
- Transient → Open Session → Persistent → Close Session → Detached

### 26. Persisting Detached Objects

- New Session - No changes will also issue UPDATE
- @Entity @Entity (selectBeforeUpdate = true) – Special Hibernate Annotation - Executes UPDATE query only if the SELECT indicates a change

### 27. Hibernate Query Language (HQL)

- More control like Pagination, Filter, etc.
- Session.createQuery(query).list()
- Session.createQuery(query).uniqueResult() with group functions such as COUNT, DISTINCT, AVG, SUM, MAX and MIN

### 28. HQL Pagination and Filtration

- setFirstResult (startPosition) – Offset
- setMaxResults (maxCount) – Limit
- SELECT new Map (key, value) FROM ClassName

### 29. Parameter Binding and SQL Injection

- Appending parameters in a simple statement will lead to SQL Injection Attack – “QUERY WHERE COLNAME =” + value – Allows attacker to inject ‘value OR 1 = 1’ – Exposes the protected and confidential data
- Use Prepared Statement with either ‘?’ or ‘:name’ – Named Parameter, get(index) or get(parameterName) respectively

### 30. Named Queries

- Best Practice – One common place for all queries related to the entity – Entity Class is the best place to group them – Reusable
- @NamedQuery (name, HQL)
- @NamedNativeQuery (name, SQL, ResultClass)
- Session.getNamedQuery(queryName)

### 31. Criteria API

- No need for SQL or HQL – Third way to extract data – Best among three – HQL uses Query no different from SQL which is complex too
- Criteria c = Session.createCriteria(Class)

### 32. Restrictions with Criteria

- Chain criterions with add methods – Criteria.add(c1).add(c2)...
- Criterion c1 = Restrictions.gt(colName, value)
- Restrictions.eq(), ge(), le(), lt(), gt(), like(), ilike(), isNull(), isNotNull(), isEmpty(), isNotEmpty and between() are some of them
- Logical Expressions are Restrictions.and(c1, c2) and Restrictions.or(c3,c4)
- Pagination methods – setFirstResult (startPosition) and setMaxResults (maxCount)
- Sorting the results – Criteria.addOrder(Order.asc/desc(fieldName))

### 33. Projections, Aggregations and Query by Example

- Criteria.setProjection(Projections.rowCount())
- property() and aggregate (group) functions such as countDistinct(), rowCount(), avg(), max(), min() and sum() are some of the projections
- Query by Example – Create Example object with default values – Bunch of properties to set in a query – Fetches the records that satisfies the property values in Example object
- Criteria.add(Example.create(exampleObject))
- Ignores null values and primary key
- Exclude the property using Example.excludeProperty(propertyName) method
- Set the property value with % and enable LIKE capability using Example.enableLike() method

### 34. Hibernate Cache

- First Level Cache (Default) – Session Cache – Mandatory - Minimizes the query (SELECT and UPDATE) execution
- Second Level Cache – Optional – Third Party Vendor
- Query Cache – Optional – Third Party Vendor

### 35. Second Level Cache

- Caching across sessions, applications and clusters
- Configurable - hibernate.cfg.xml – cache.provider\_class – Determined by Concurrency Strategies - EHCacheProvider/OSCacheProvider/SwarmCacheProvider/JBossCacheProvider
- Turn on the second level cache (optional) – cache.use\_second\_level\_cache – true
- Annotations - @Entity @Cacheable @Cache(usage=CacheConcurrencyStrategy.NONE, TRANSACTIONAL, READ\_WRITE, NON\_STRICT\_READ\_WRITE, READ\_ONLY)

### 36. Query Cache

- HQL and Criteria are stored in a different memory location – Second Level Caching mechanism will not cache them
- Configurable - hibernate.cfg.xml – cache.use\_query\_cache – true
- Query.setCacheable(true) or Criteria.setCacheable(true) for different sessions
- Use them with care – End up querying a lot of data – Cache something not required - Caching them make the system inefficient

# Chapter 25 – iBatis

## *iBatis*

---

ORM - object model is not in sync with data model - objects are mapped to result set of a query or stored procedure.

### **JDBC Framework**

- developers write SQL
- no try/catch/finally

### **SQL Mapper**

- automatically maps object properties to prepared statement parameters
- automatically maps result sets to objects
- support for getting rid of N+1 queries

### **Transaction Manager**

- provides transaction management for database operations
- support transaction management (Spring, EJB CMT)

### **Great integration with Spring**

It's an ORM

- does not generate SQL
- does not have a proprietary query language
- does not know about object identity
- does not transparently persist objects
- does not build an object cache

## Chapter 26 – Framework Configuration and Integration

### *Framework Configuration Files*

---

#### **JSF**

Config File: faces-config.XML

#### **Struts**

Config File: struts-config.XML

#### **Spring**

Config File: applicationContext.XML

#### **Hibernate**

Config File: hibernate.cfg.XML

Mapping File: xxx.hbm.XML

#### **iBatis**

Config File: SqlMapConfig.XML

Mapping File: SqlMap.XML

#### **Web Application**

Config File: web.XML

### *Framework Integration*

---

#### **Struts+Spring+Hibernate (Web Framework+ DI Framework + ORM Framework)**

Web Page -> DI -> DAO -> DB

1+2 = Spring's Struts Plug-in "ContextLoaderPlugin"

- Action class extends Spring's ActionSupport class

- Get Spring bean via getWebApplicationContext()

- struts-config.XML

- spring-web.jar & spring-struts.jar

2+3= Spring's "LocalSessionFactoryBean"

applicationContext.XML

- Spring's DriverManagerDataSource, DataSource.XML & PropertyPlaceholderConfigurer, DB.properties

- Spring's HibernateSessionFactory.XML

- DAO class extends HibernateDAOSupport class

### **JSF+Spring+Hibernate (Web Framework+ DI Framework + ORM Framework)**

1+2= faces-config.XML

- SpringBeanFacesELResolver
- web.XML
- Spring's ContextLoadListener
- Spring's RequestContextListener
- JSF's FacesServlet

### **JSF+Spring+iBatis (Web Framework+ DI Framework + ORM Framework)**

2+3=Minimum Three Configuration files + web.XML

1. Spring Config -

applicationContext.XML

Spring's SqlMapClientFactoryBean

Instance of ClassPathXmlApplicationContext(applicationContext.XML).getBean(beanName)

2. SQL Map Config - SqlMapConfig.XML

SqlMapClient.queryForObject(sqlName, input) or update(same)

3. SQL Map - domainObject.XML

### **Struts+Hibernate (Web Framework + ORM Framework)**

- Create Hibernate Struts Plug-in
- Get Hibernate Session Factory
- Store it into the Servlet context
- Include it in struts-config.XML
- <plug-in>

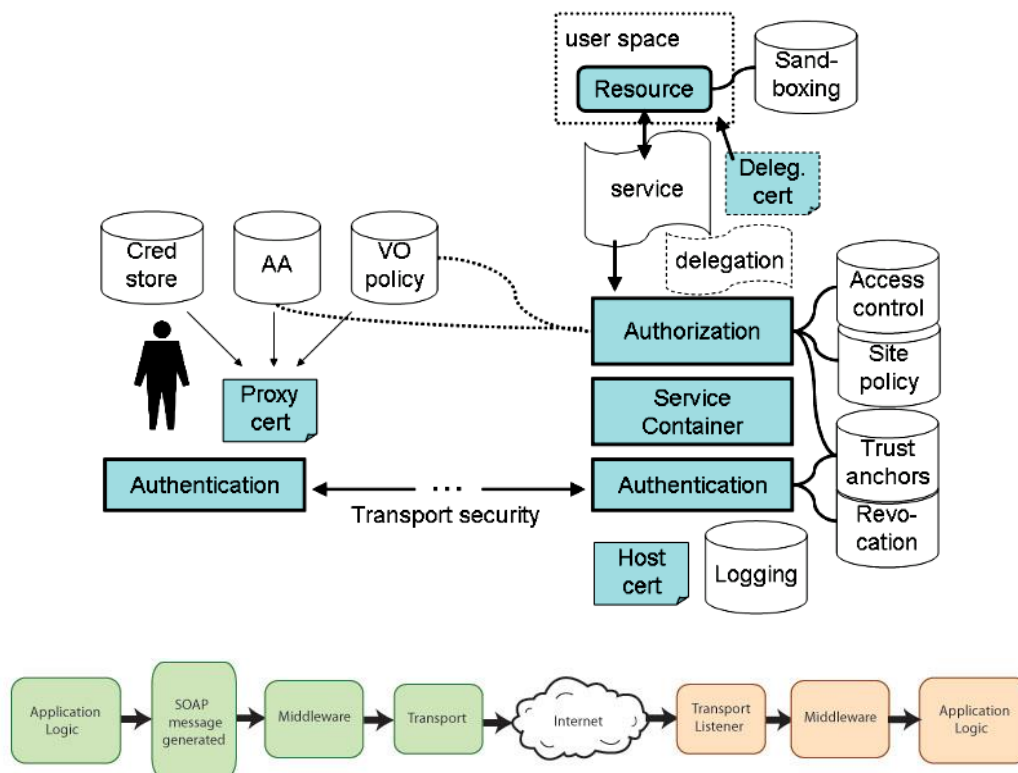
Implements Plugin

Init(ActionServlet, ModuleConfig)

# Chapter 27 – Apache Axis

## Apache Axis

Apache Axis is an execution of Simple Object Access Protocol also called as SOAP submission to W3C. SOAP is a lightweight protocol meant for the purpose of exchange of controlled information in a distributed and decentralized environment. It is a protocol based on Extensible Markup Language, which consists of three parts: a standard for representing remote procedure responses and calls, a combination of encoding rules to express examples of application-defined data types and an envelope defining a structure for describing the content of a message and the method of processing it.



## Apache Axis 2

---

The Apache Axis2 project is a Java-based implementation of both the client and server sides of the Web services equation. Designed to take advantage of the lessons learned from Apache Axis 1.0, Apache Axis2 provides a complete object model and a modular architecture that makes it easy to add functionality and support for new Web services-related specifications and recommendations.

- complete redesign and rewrite
- more efficient, more modular and more XML-oriented
- easy addition of Plugin "modules" such as Security and Reliability

Axis2 enables you to easily perform the following tasks:

1. Send SOAP messages
2. Receive and process SOAP messages
3. Create a Web service out of a plain Java class
4. Create implementation classes for both the server and client using WSDL
5. Easily retrieve the WSDL for a service
6. Send and receive SOAP messages with attachments
7. Create or utilize a REST-based Web service
8. Create or utilize services that take advantage of the WS-Security, WS-ReliableMessaging, WS-Addressing, WS-Coordination, and WS-Atomic Transaction recommendations
9. Use Axis2's modular structure to easily add support for new recommendations as they emerge

### Enhancements

Apache Axis2 not only supports SOAP 1.1 and SOAP 1.2, but it also has integrated support for the widely popular REST style of Web services. The same business logic implementation can offer both a WS-\* style interface as well as a REST/POX style interface simultaneously.

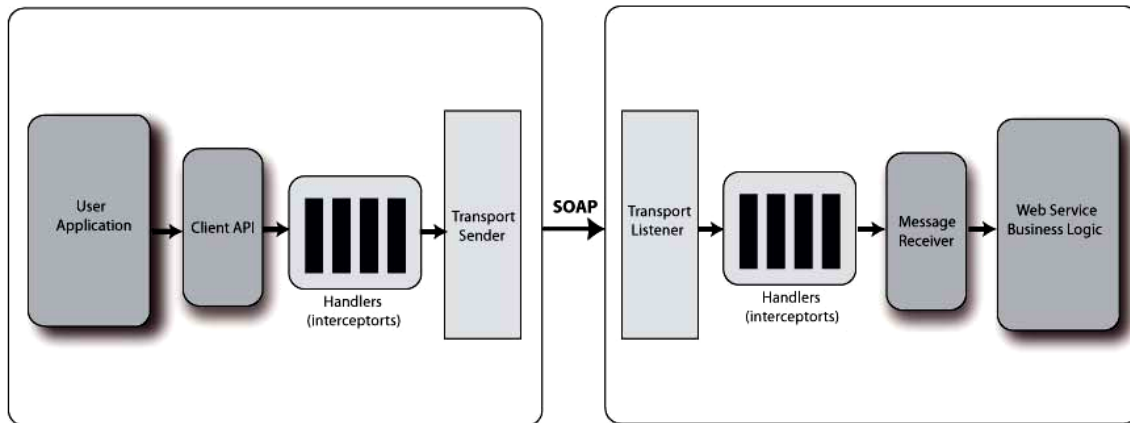
Axis2 has support for Spring Framework

Axis2 comes with many new features, enhancements and industry specification implementations. The key features offered are as follows:

- Speed
- Low memory foot print
- AXIOM - **AXIS2** comes with its own light-weight **Object Model**
- Hot Deployment
- Asynchronous Web services
- Message Exchange Patterns (MEPs) Support
- Flexibility
- Stability
- Component-oriented Deployment
- Transport Framework
- WSDL support - supports version 1.1 and 2.0
- Add-ons - WSS4J for security (Apache Rampart), Sandesha for reliable messaging, Kandula which is an encapsulation of WS-Coordination, WS-AtomicTransaction and WS-BusinessActivity.
- Composition and Extensibility

## How Axis2 Handles SOAP Messages

Axis2 can handle processing for both the sender and the receiver in a transaction. From the Axis2 perspective, the structure looks like this:



On each end, you have an application designed to deal with the (sent or received) messages. In the middle, you have Axis2, or rather, you can have Axis2. The value of Web services is that the sender and receiver (each of which can be either the server or the client) don't even have to be on the same platform, much less running the same application. Assuming that Axis2 is running on both sides, the process looks like this:

The sender creates the SOAP message.

Axis "handlers" perform any necessary actions on that message such as encryption of WS-Security related messages.

The transport sender sends the message.

On the receiving end, the transport listener detects the message.

The transport listener passes the message on to any handlers on the receiving side.

Once the message has been processed in the "pre-dispatch" phase, it is handed off to the dispatchers, which pass it on to the appropriate application.

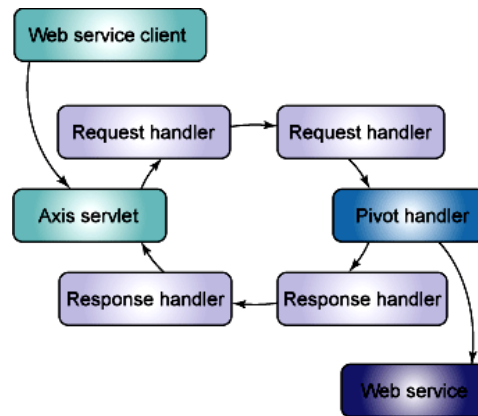
In Axis2, these actions are broken down into "phases", with several pre-defined phases, such as the "pre-dispatch", "dispatch," and "message processing", being built into Axis2. Each phase is a collection of "handlers". Axis2 enables you to control what handlers go into which phases, and the order in which the handlers are executed within the phases. You can also add your own phases and handlers.

Handlers come from "modules" that can be plugged into a running Axis2 system. These modules, such as Rampart, which provides an implementation of WS-Security, and Sandesha, which provides an implementation of WS-ReliableMessaging, are the main extensibility mechanisms in Axis2.

## Axis Flow Architecture

Apache's first SOAP implementation relied on the concept of a provider. This bit of code decoded the object and method from a SOAPEnvelope, instantiated the local object, and returned the result to be wrapped up in a return SOAP envelope. Axis has a provider equivalent, but often it is referred to as the pivot point. This is because Axis provides an architecture that you can use to define transaction flows. You define the flows through handlers. Each handler is able to act on the current incoming and outgoing SOAP

envelope. Handlers provide the ability to perform methods and services as part of a flow for a larger service, as shown in Figure 1. A pivot point is just another handler, but it indicates the point at which the request becomes a response. The pivot point is one place where you can insert your cache.



## Message Exchange Patterns (MEPs)

Although all SOAP messages carry the same structure, the ways in which they are used can be combined into a number of different "message exchange patterns", or MEPs. The two major message exchange patterns are:

**In-Out:** in this MEP, the client sends a SOAP message to the server, which processes the message and sends a response back. This is probably the most commonly used MEP, and is useful for tasks such as searching for information or submitting information in situations in where acknowledgment is important.

**In-Only:** In this MEP, the client sends a message to the server without expecting a response. You may use this MEP for activities such as pinging a server to wake it up, reporting logging information for which you do not need an acknowledgment and so on.

Within these two MEPs, you also have several variables to consider:

**Blocking versus non-blocking:** When the client sends a message, the application may wait to receive a response before moving on, or it may simply send a message and move on by specifying a callback action to be completed when the response is received.

**Number of parameters:** Ultimately, a message sent from a client to server is intended to execute a particular action. That action may not require any parameters, or it may require one or more parameters. These parameters must be encoded as part of the payload of the message.

Taking all these options into consideration, you can create virtually any MEP. For example, you can create an Out-Only system by reversing roles for the In-Only MEP. Apache Axis2 also includes support for less prominent MEPs, such as Robust-In-Only.

## Choosing a Client Generation Method

---

Axis2 gives you several options when it comes to mapping WSDL to objects when generating clients. Three of these options are Axis2 Data Binding Framework, XMLBeans, and JiBX data binding. All of these methods involve using data binding to create Java objects out of the XML structures used by the service, and each has its pros and cons. You can also generate XML in-out stubs that are not based on data binding.

### Axis2 DataBinding Framework (ADB)

ADB is probably the simplest method of generating an Axis2 client. In most cases, all of the pertinent classes are created as inner classes of a main stub class. ADB is very easy to use, but it does have limitations. It is not meant to be a full schema binding application, and has difficulty with structures such as XML Schema element extensions and restrictions.

### XMLBeans

Unlike ADB, XMLBeans is a fully functional schema compiler, so it doesn't carry the same limitations as ADB. It is, however, a bit more complicated to use than ADB. It generates a huge number of files, and the programming model, while being certainly usable, is not as straightforward as ADB.

### JiBX

JiBX is a complete data binding framework that actually provides not only WSDL-to-Java conversion, as covered in this document, but also Java-to-XML conversion. In some ways, JiBX provides the best of both worlds. JiBX is extremely flexible, enabling you to choose the classes that represent your entities, but it can be complicated to set up. On the other hand, once it is set up, actually using the generated code is as easy as using ADB.

In the end, for simple structures, ADB will likely be enough for you. If, on the other hand you need more power or flexibility, whether you choose XMLBeans or JiBX depends on how much power or flexibility you need and your tolerance for complexity.

## Choosing a Service Creation Method

---

Axis2 provides a number of ways to create a service, such as:

- Create a service and build it from scratch. In this case, you build your service class to specifically access AXIOM OMElement objects, then create the services.xml file and package it for deployment.
- Deploy Plain Old Java Objects (POJOs) as a service.
- Generate the service from WSDL. Just as you can generate clients with WSDL, you can also generate the skeleton of a service.

Writing client using Apache SOAP API:

```
Service service = new Service();  
Call call = (Call)service.createCall();  
//Set the target service host and service location  
call.setTargetEndpointAddress(new URL(http://localhost:8080/axis/services));
```

```
//Invoke the operation
call.setOperationName(new QName("EchoService", "echo"));
call.addParameter("in0", XMLType.XSD_STRING, ParameterMode.IN);
call.setReturnType(XMLType.XSD_STRING);
Object ret = call.invoke(new Object[]{in0});
//The object ret will either be a String on success or RemoteException on failure.
```

Four Client files generated by WSDL2Java:

1. Echo.java
2. EchoService.java
3. EchoServiceLocator.java
4. EchoSoapBindingStub.java

Writing client using generated stubs:

```
//Create Echo client object and point to the service you want to use
Echo echo = new EchoServiceLocator().getEcho(new URL(http://server:port/axis/services/Echo));
//Invoke the method as if local
String backTalk = echo.echo("Hi");
```

# Chapter 28 – Jersey

## Jersey Framework

Jersey framework is more than the JAX-RS Reference Implementation. Jersey provides its own API that extends the JAX-RS toolkit with additional features and utilities to further simplify RESTful service and client development.

### Jersey Annotations (javax.ws.rs)

| Annotation                                          | Description                                                                                       | Usage                                                                                                                                                                       |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @GET                                                | Indicates that the following method will answer to an HTTP GET/Retrieve request.                  | <b>@GET</b><br>public String getHTML() {<br>...<br>}                                                                                                                        |
| @POST                                               | Indicates that the following method will answer to an HTTP POST/Create request.                   | <b>@POST</b><br>@Consumes("application/json")<br>@Produces("application/json")<br>public RestResponse<Contact> create(Contact contact) {<br>...<br>}                        |
| @PUT                                                | Indicates that the following method will answer to an HTTP PUT/Update request.                    | <b>@PUT</b><br>@Consumes("application/json")<br>@Produces("application/json")<br>@Path("{contactId}")<br>public RestResponse<Contact> update(Contact contact) {<br>...<br>} |
| @DELETE                                             | Indicates that the following method will answer to an HTTP DELETE/Remove request.                 | <b>@DELETE</b><br>@Produces("application/json")<br>@Path("{contactId}")<br>public RestResponse<Contact> delete(@PathParam("contactId") int contactId) {<br>...<br>}         |
| @Consumes (MediaType.APPLICATION_XML[, more-types]) | 'Consumes' defines which MIME type is consumed by a method annotated with @POST, @PUT or @DELETE. | @PUT<br><b>@Consumes("application/json")</b><br>@Produces("application/json")<br>@Path("{contactId}")<br>public void delete(Contact contact) { ...<br>}                     |

|                                                                               |                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@Produces</code><br>( <code>MediaType.TEXT_PLAIN</code> [, more-types]) | 'Produces' defines which MIME type is delivered or produced by a method annotated with <code>@GET</code> . The standard outputs are "text/plain", "text/html", "application/xml" and "application/json".                                      | <code>@GET</code><br><b><code>@Produces("application/json")</code></b><br>public Contact getJSON() {<br>...<br>}                                                                                                                                                                                                                                                       |
| <code>@Path(your_path)</code>                                                 | Specifies the URL path on which this method will be invoked. The base URL is based on your application name, the servlet and the URL pattern from the web.xml configuration file.                                                             | <code>@GET</code><br><code>@Produces("application/xml")</code><br><b><code>@Path("users/{username: [a-zA-Z][a-zA-Z_0-9]*}")</code></b><br>public Contact getXML() {<br>...<br>}                                                                                                                                                                                        |
| <code>@PathParam</code>                                                       | Used to inject values from the URL into a method parameter. We can bind REST-style URL parameters to method arguments using <code>@PathParam</code> annotation.                                                                               | <code>@GET</code><br><code>@Produces("application/xml")</code><br><code>@Path("xml/{firstName}")</code><br>public Contact<br>getXML( <b><code>@PathParam("firstName") String</code></b> firstName) {<br>Contact contact =<br>contactService.findByName(firstName);<br>return contact;<br>}                                                                             |
| <code>@QueryParam</code>                                                      | Request parameters in query string can be accessed using <code>@QueryParam</code> annotation.                                                                                                                                                 | <code>@GET</code><br><code>@Produces("application/json")</code><br><code>@Path("json/companyList")</code><br>public CompanyList<br>getJSON( <b><code>@QueryParam("start") int</code></b> start,<br><b><code>@QueryParam("limit") int</code></b> limit) {<br>CompanyList list = new<br>CompanyList(companyService.listCompanies(start,<br>limit));<br>return list;<br>} |
| <code>@FormParam</code>                                                       | The REST resources will usually consume XML/JSON for the complete Entity Bean. Sometimes, you may want to read parameters sent in POST requests directly using <code>@FormParam</code> .                                                      | <code>@POST</code><br>public String save( <b><code>@FormParam("firstName")</code></b><br><b><code>String firstName,</code></b><br><b><code>@FormParam("lastName") String</code></b> lastName) {<br>...<br>}                                                                                                                                                            |
| <code>@HeaderParam</code>                                                     | Binds the value(s) of a HTTP header to a resource method parameter, resource class field, or resource class bean property.                                                                                                                    | public class MyBeanParam {<br><b><code>@HeaderParam("header")</code></b><br>private String headerParam;<br>}                                                                                                                                                                                                                                                           |
| <code>@CookieParam</code>                                                     | Binds the value of a HTTP cookie to a resource method parameter, resource class field, or resource class bean property.                                                                                                                       | public class MyBeanParam {<br><b><code>@CookieParam("cookieName")</code></b><br>private String cookieParam;<br>}                                                                                                                                                                                                                                                       |
| <code>@MatrixParam</code>                                                     | Binds the value(s) of a URI matrix parameter to a resource method parameter, resource class field, or resource class bean property.                                                                                                           | public class MyBeanParam {<br><b><code>@MatrixParam("m")</code></b><br>@Encoded<br>@DefaultValue("default")<br>private String matrixParam;<br>}                                                                                                                                                                                                                        |
| <code>@BeanParam</code>                                                       | It can contain all parameter injections. The JAX-RS runtime will instantiate the object and inject all its fields and properties annotated with either one of the <code>@PathParam</code> annotation or the <code>@Context</code> annotation. | public class MyBean {<br><b><code>@FormParam("myData")</code></b><br>private String data;<br><b><code>@HeaderParam("myHeader")</code></b><br>private String header;<br><b><code>@PathParam("id")</code></b><br>public void setResourceid(String id) {...}<br>}                                                                                                         |

|                |                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                |                                                                                                                                                                                                                                                                     | <pre>@Path("myresources") public class MyResources {     @POST     @Path("{id}")     public void post(@BeanParam MyBean myBean) {...} }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| @DefaultValue  | Defines the default value of request meta-data that is bound using either one of PathParam, QueryParam, MatrixParam, CookieParam, FormParam or HeaderParam.                                                                                                         | <pre>@Path("{id:\\d+}") public class InjectedResource {     // Injection onto field     @DefaultValue("q") @QueryParam("p")     private String p; }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| @Encoded       | Disables automatic decoding of parameter values bound using QueryParam, PathParam, FormParam or MatrixParam.                                                                                                                                                        | <pre>public class MyBeanParam {     @MatrixParam("m")     @Encoded     @DefaultValue("default")     private String matrixParam; }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| @Context       | This annotation is used to inject information into a class field, bean property or method parameter. When deploying a JAX-RS application using servlet then ServletConfig, ServletContext, HttpServletRequest and HttpServletResponse are available using @Context. | <pre>@GET public String get(@Context UriInfo ui) {     MultivaluedMap&lt;String, String&gt; queryParams = ui.getQueryParameters();     MultivaluedMap&lt;String, String&gt; pathParams = ui.getPathParameters(); }  @GET public String get(@Context HttpHeaders hh) {     MultivaluedMap&lt;String, String&gt; headerParams = hh.getRequestHeaders();     Map&lt;String, Cookie&gt; pathParams = hh.getCookies(); }  @Context public void setRequest(Request request) {     String[] requestParams = request.getRequestParams(); }  public MySingletonResource(@Context SecurityContext securityContext) {     ... }</pre> |
| @Singleton     | In this scope there is only one instance per jax-rs application.                                                                                                                                                                                                    | <pre>@Singleton @Path("/printers") public class PrintersResource {     ... }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| @RequestScoped | Default lifecycle (applied when no annotation is present). In this scope the resource instance is created for each new request and used for processing of this request.                                                                                             | <pre>@RequestScoped @Path("/pathUrl") public class UserAction {     ... }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| @PerLookup     | In this scope the resource instance is created every time it is needed for the processing even it handles the same request.                                                                                                                                         | <pre>@PerLookup @Path("/pathUrl") public class GranularTransaction {     ... }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |